



МИНОБРНАУКИ РОССИИ
федеральное государственное бюджетное образовательное
учреждение высшего образования
«Национальный исследовательский университет «МЭИ»

Институт
Кафедра

ИВТ
ВМСС

ВЫПУСКНАЯ КВАЛИФИКАЦИОННАЯ РАБОТА
(бакалаврская работа)

Направление 09.03.01 Информатика и вычислительная техника
(код и наименование)

Направленность (профиль) Вычислительные машины, комплексы,
системы и сети

Форма обучения очная
(очная/очно-заочная/заочная)

Тема: Разработка имитационной модели эволюции простейших
организмов

Студент А-07-16 Голикова А.В.
группа подпись фамилия и инициалы

Научный к.т.н. доцент Раскатова М.В.
руководитель уч. степень должность подпись фамилия и инициалы

Консультант уч. степень должность подпись фамилия и инициалы

Консультант уч. степень должность подпись фамилия и инициалы

«Работа допущена к защите»

Зав. кафедрой к.т.н. доцент Вишняков С.В.
уч. степень звание подпись фамилия и инициалы

Дата _____

Москва, 2020

Аннотация

Выпускная работа студентки Голиковой А.В. группы А-07-16.

Работа посвящена разработке приложения, реализующего имитационную модель эволюции простейших существ с использованием сред разработки Unity и Visual Studio.

Работа представлена на 56 страницах, 6 таблицах, 27 рисунках и 1 приложении.

Graduation work of the student A. Golikova group A-07-16.

The work is devoted to the development of an application that implements a simulation model of the evolution of protozoan using the Unity and Visual Studio development environments.

The work is presented on 56 pages, 6 tables, 27 figures and 1 supplement.

Содержание

Аннотация.....	2
Определения	4
Введение	6
1. Постановка задачи.....	8
1.1. Анализ предметной области.....	8
1.2. Сравнение с существующими аналогами	10
1.3. Формулировка задачи	15
2. Разработка приложения	17
2.1. Описание средств разработки	17
2.2. Выбор примитивов для отображения элементов системы	17
2.3. Разработка структуры приложения.....	19
2.4. Расчет компонентов симуляции	25
2.5. Разработка алгоритмов	29
2.4. Разработка пользовательского интерфейса	42
3. Реализация и тестирование программы	47
3.1. Реализация программы	47
3.2. Тестирование	49
Заключение	51
Список использованной литературы.....	52
Приложение А. Техническое задание	53
Приложение Б. Листинг программы	56

Определения

В настоящем проекте применяются следующие термины:

1. Сцена Unity — отдельный файл, содержащий набор объектов, сценариев и настроек [1].
2. GameObject (игровой объект) - базовый класс для всех сущностей в сценах Unity. Сами по себе объекты невидимы и не обладают ничем, кроме имени. Их поведение определяется компонентами. Разные компоненты отвечают за разные аспекты; например, визуализаторы мешей (Mesh Renderers) отображают трехмерные меши (сетки), а источники звука (Audio Sources) воспроизводят звуки[2]. Свои компоненты можно создавать, используя скрипты.
3. Скрипт - отдельный класс, который взаимодействует с внутренними механизмами Unity за счет создания класса, наследующего от встроенного класса, называемого MonoBehaviour. Скрипт определяет только план компонента и, таким образом, никакой его код не будет активирован до тех пор, пока экземпляр скрипта не будет присоединен к игровому объекту[3].
4. Игровой день - отрезок времени длиной 10 секунд. Используется для обозначения одних суток симуляции.
5. Эволюция - биологический процесс, в результате которого меняется генетическая структура организмов, происходит образование новых видов и вымирание старых. Основные факторы эволюции - мутации и естественный отбор.
6. Мутации - стойкое (то есть такое, которое может быть унаследовано потомками данной клетки или организма) изменение генома[4].
7. Естественный отбор - основной фактор эволюции, в результате действия которого в популяции увеличивается число особей, обладающих более высокой приспособленностью (наиболее благоприятными признаками), в то время как количество особей с неблагоприятными признаками уменьшается[5].

8. Геном — совокупность наследственного материала, заключенного в клетке организма[6].

Введение

В настоящее время происходит информатизация в сфере образования - школы оборудуют интерактивными досками, компьютерами, разрабатываются электронные учебники. На уроках информатики дети отправляют решения задач по программированию на специально разработанный для этого сайт.

Применение новейших технологий в обучении позволяет значительно улучшить образование. Учащиеся овладевают материалом более полно и всесторонне при наглядной демонстрации объекта обучения и возможности взаимодействовать с ним. Это является основой интерактивного обучения - сравнительно новой (первые элементы начали разрабатываться в 70-80х годах) формы организации познавательной деятельности. Само понятие "интерактивное обучение" в педагогической науке еще формируется и уточняется, но принято понимать его как обучение, основанное на прямом взаимодействии учащихся между собой или с учебным окружением, где центральным источником познания является личный опыт учащегося[7].

Такой вид обучения позволяет учителю заинтересовать школьников, повысить их мотивацию к обучению (в том числе и мотивацию самостоятельно изучать данный предмет вне рамок школьного курса).

Разрабатываемое приложение будет использоваться на уроках биологии или для домашнего обучения. Кроме того, программа может использоваться для моделирования в научно-популярных видеозаписях об эволюции.

Приложение имеет тематику, связанную с разделом эволюционной биологии. Этот раздел изучает происхождение видов от общих предков, наследственность и изменчивость их признаков, размножение и разнообразие форм в ходе эволюционного развития. Соответственно, в приложении будет показано, как от одного вида организмов может образоваться несколько новых из-за воздействия мутаций (таких, например, как мутация, дающая возможность травоядному существу переваривать мясо, из которого

можно извлечь больше энергии, но она влияет на эффективность получения энергии из овощей).

В первом разделе данной работы раскрывается значение термина "имитационное моделирование", происходит анализ аналогичных приложений и постановка задачи.

Во втором разделе разрабатываются структура основные алгоритмы программы и пользовательский интерфейс.

В третьем разделе реализуется и тестируется программа.

1. Постановка задачи

1.1. Анализ предметной области

Существует множество процессов, которые нельзя описать аналитической моделью. Причиной этому может являться, например, наличие случайных переменных или иные нелинейности. Примером такого процесса может являться эволюция. Провести эксперимент над реальными существами дорого и трудоемко, поэтому логичнее использовать математическую модель. Разработать аналитическую модель эволюции практически невозможно по многим причинам. Одна из причин - эволюция происходит во времени и зависит от случайных мутаций.

Для построения модели такого процесса прибегают к имитационному моделированию. Это метод исследования, при котором изучаемую систему заменяют моделью, описывающей ее с достаточной точностью. Имитационное моделирование обеспечивает возможность испытания, оценки и проведения экспериментов с исследуемой системой без каких-либо непосредственных воздействий на нее. Результаты экспериментов будут зависеть от случайных процессов в симуляции. Для получения устойчивой статистики эксперимент следует проводить несколько раз.

Имитационная модель – универсальное средство исследования сложных систем, представляющее собой логико-алгоритмическое описание поведения отдельных элементов системы и правил их взаимодействия, отображающих последовательность событий, возникающих в моделируемой системе[8].

Такие модели применяются для многих целей, начиная от научных исследований, заканчивая демонстрациями учащимся для углубленного понимания процессов, происходящих в системе.

Одним из самых наглядных и простых для понимания примеров эволюции является эволюция простейших - одноклеточных существ, представляющих собой полноценный организм с возможными мутациями.

Простейшие - царство одноклеточных живых организмов, которые содержат одно или несколько ядер.

Формы и окраска простейших отличаются значительным разнообразием. Многие имеют скелетные структуры или тела, окрашенные пигментами, но огромное количество простейших имеет практически бесцветное тело; кроме того, они лишены скелетов и панцирей. Размеры варьируются от 0,01мм до нескольких миллиметров.

Обитают простейшие в основном во влажной среде - морской и пресной воде, других организмах, влажной почве.

Одноклеточные животные являются одним из звеньев в пищевых цепях биосферы. Питаются они готовыми органическими веществами - бактериями, другими одноклеточными организмами или водорослями.

Простейшие обладают таким свойством, как раздражимость - способностью реагировать на изменения окружающей среды - передвигаться от источника раздражения или к нему. Размножаются преимущественно делением. При этом из одной клетки получается две или больше.

Наука, изучающая простейших, называется протозоологией. Бурный ее рост не случаен и определяется рядом важных причин:

- Простейшие являются существенным компонентом биосферы и необходимым звеном в цепи ее обмена веществ.
- Простейшие представляют особый уровень развития живого мира, поэтому их изучение имеет общебиологическое значение.
- Простейшие ныне используются как модель эукариотной клетки для исследований в области молекулярной биологии, генетики, биохимии, радиобиологии и т.д.
- Патогенные, почвенные и другие простейшие имеют важное народнохозяйственное значение[9].

1.2. Сравнение с существующими аналогами

Так как разрабатывается имитационная модель, в первую очередь необходимо рассмотреть аналоги, которые являются симуляторами. Из-за того, что аналогичных приложений достаточно мало, имеет смысл рассмотреть еще и игры, которые основаны на процессе эволюции одноклеточных существ.

Одним из самых известных аналогов является симулятор эволюции одноклеточных существ "The strongest survives". Данный симулятор выполнен в формате 2D - у него имеются лишь два измерения. Его интерфейс изображен на рис.1. Клетки имеют равные возможности в передвижении, различаются лишь стратегии перемещения. Каждая клетка может передвигаться в четырех направлениях. Если в выбранном направлении находится еда, яд или мертвая клетка, то она получает энергию от них и перемещается. Также клетки могут взаимодействовать с другими клетками - в случае одинакового генома обе клетки получают энергию, в случае различного - клетка, сделавшая ход, забирает часть энергии у другой. На выбор направления влияет геном клетки. Например, клетка с высоким уровнем параметра "агрессия" с большей вероятностью пойдет в направлении клетки с чужеродным геномом. Аналогично происходит и для других параметров - голода, коллективности, тяги к яду и тяги к мертвым клеткам. Параметр "репродуктивность" влияет на количество энергии, необходимое клетке для размножения, но базовый уровень задается пользователем. При размножении есть вероятность мутации. Цвет клетки зависит от генома.

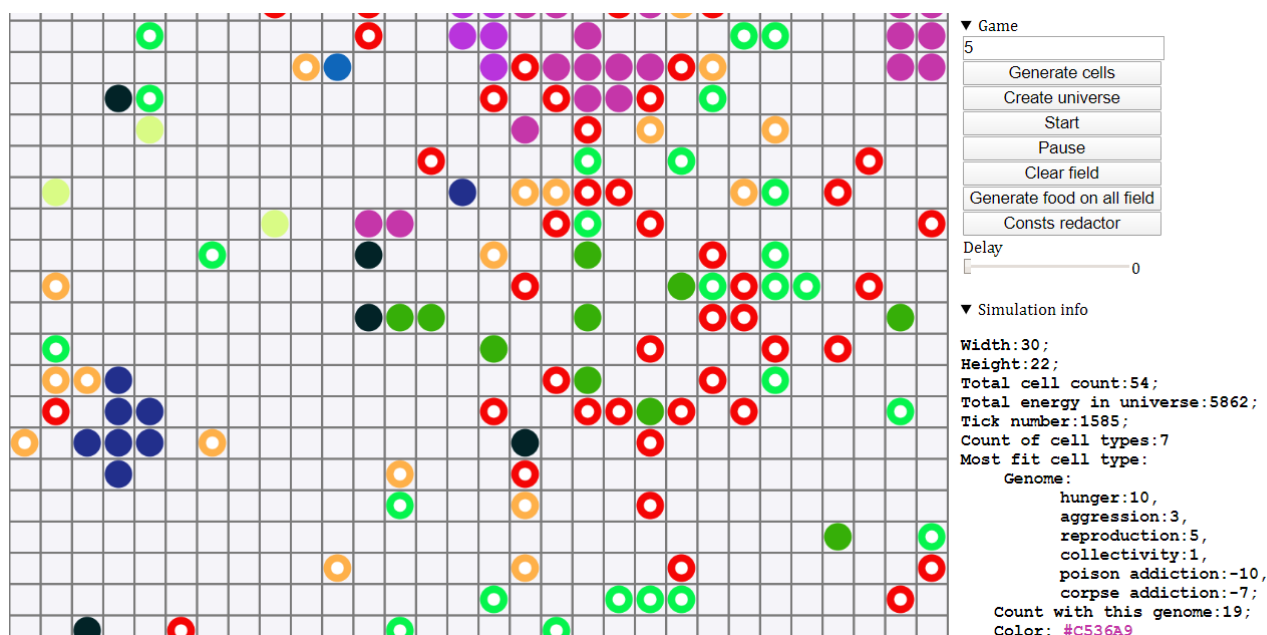


Рис.1. Интерфейс "The strongest survives"

Симулятор "The strongest survives" можно назвать полноценной математической моделью эволюции, так как присутствуют основные её параметры - естественный отбор (выживают только приспособленные клетки) и мутации.

У симулятора присутствуют и недостатки. При запуске симуляции с отсутствием какой-либо еды клетки могут начать вырабатывать энергию из ниоткуда за счет коллективности. Отсутствие у клеток различий в физических свойствах также является недостатком. Кроме того, можно считать недостатком и то, что параметры клеток не влияют на энергию, что может привести к появлению "суперклетки", невозможной в реальных условиях.

В качестве игрового аналога можно рассматривать один из этапов игры "Spore" - этап "клетка". Эта игра выполнена в 3D-формате, что делает графическую составляющую более приятной и интересной для человека. Основная её цель - развить свое существо от одноклеточного организма до уровня космической цивилизации, улучшая его так, чтобы пройти естественный отбор.

На этапе "клетка" игрок управляет одноклеточным существом, собирая растения или кусочки мяса и получая за это очки, называемые "очки ДНК". В

игре присутствует возможность взаимодействовать с другими клетками - наносить им урон. При нанесении достаточного количества урона другая клетка погибает, оставляя после себя кусочки мяса. Если была убита клетка, имеющая фрагмент, которого еще нет в редакторе существ, представленном на рис.3, появится пиктограмма этого фрагмента. Слишком маленькие клетки поглощаются сразу, без необходимости наносить урон, со слишком большими клетками взаимодействовать нельзя. На рис.2 можно увидеть существо игрока, другие клетки и кусочки еды.



Рис.2. Этап "клетка" игры Spore

При наличии достаточного количества "очков ДНК" игрок может изменить свое существо. При этом начальное существо откладывает икринку, а игрок переходит в редактор существ, изображенный на рис.3. В редакторе можно обменивать "очки ДНК" на новые конечности для своего существа или убрать старые конечности. После завершения редактирования пользователь продолжает игру за новое существо, вылупившееся из икринки. Вместе с существом игрока изменяется и вся популяция этого существа. Это можно считать отображением мутаций, так как происходит изменение генома, которое может быть унаследовано.



Рис.3. Редактор существ в Spore.

При достижении определенного количества "очков ДНК" (включая потраченные на новые конечности) существо увеличивается в размерах. Размер других клеток не изменяется. Слишком маленькие относительно существа игрока клетки исчезают. Крупные существа становятся меньше, появляется возможность взаимодействовать с ними. Эти клетки имеют больше фрагментов, чем клетки меньшего размера, соответственно, их способности значительно выше. Некоторые организмы имеют большую скорость, другие развили способность отравлять, колоть шипами или атаковать электричеством.

Несмотря на наличие отображения мутаций у существа игрока, этап "клетка" не может считаться полноценной математической моделью, так как остальные клетки остаются неизменными и не эволюционируют. При росте игрока появляются новые улучшенные типы клеток, но это совершенно новые клетки, их эволюции не наблюдается.

Также нет возможности увидеть естественный отбор, так как популяция всех организмов остается постоянной. При исчезновении клеток, не являющихся клеткой игрока, появляются точно такие же.

Основные преимущества и недостатки аналогов приведены в табл.1.

Табл.1.

Сравнение с существующими аналогами

Название аналога	Преимущества	Недостатки
The strongest survives	Стратегия поведения клетки зависит от ее генома	Клетки не имеют различий в физических свойствах, скорость всех клеток одинакова
	Бесплатная	Мутации дают только преимущества, что ведет к возможному появлению "суперклетки"
	Множество различных параметров, влияющих на результат симуляции	При запуске симуляции с нулевым количеством пищи клетки начинают вырабатывать энергию "из ниоткуда"
Spore	Трехмерная анимация, позволяющая значительно улучшить графическую составляющую игры	Сравнительно высокая стоимость
	Клетки имеют различия в физических свойствах	Отсутствие естественного отбора
		Отсутствие мутаций у неигровых клеток

В результате анализа недостатков и преимуществ аналогов можно сделать вывод о том, что имитационную модель следует разрабатывать в формате 3D - это улучшит внешний вид программы. Также необходимо не дать возможность клеткам получать какие-либо преимущества без затрат энергии и учесть физические особенности клеток - скорость и размер.

1.3. Формулировка задачи

Необходимо разработать приложение, реализующее имитационную модель эволюции простейших организмов.

Модель должна отражать основные свойства эволюции - мутации и естественный отбор. Для создания последнего необходима конкуренция между организмами, например, за еду.

Соответственно, имитационная модель должна обладать следующими функциями:

- Возможность организмов находить и получать пищу.

- Размножение приспособленных организмов и смерть неприспособленных.

Под приспособленностью подразумевается способность добыть достаточное количество энергии для жизнедеятельности и размножения.

- Мутации организмов.

- Возможность пользователя задавать различные параметры симуляции для получения более разнообразных результатов (отражение процесса эволюции в различных условиях).

Основными изменяемыми свойствами одноклеточного организма для создания его математической модели будут являться скорость, размер, чувствительность (расстояние, на котором клетка может заметить еду) и уровень хищничества (влияет на эффективность получения энергии из растительной пищи или мяса).

Клетка должна иметь следующие возможности:

- "Ощущать" присутствие еды или других клеток на определенном расстоянии от себя.

- Передвигаться в определенном направлении в зависимости от информации, полученной от органа чувств.

- Взаимодействовать с пищей, уничтожая ее и получая определенное количество энергии в зависимости от типа пищи и того, насколько эффективно данная клетка может переваривать этот тип пищи. Кроме того, клетки с

максимальным уровнем параметра "хищничество" не должны есть растительную пищу, а клетки с минимальным уровнем не должны есть мясо, так как они неспособны получить из них энергию.

-В конце каждого игрового дня организмы, имеющие достаточное количество энергии должны размножиться, при этом каждый организм должен создавать одну дочернюю клетку.

-При создании дочерней клетки должна быть вероятность изменения одного или нескольких ее признаков в качестве отображения мутаций.

Основными параметрами симуляции, на которые может влиять пользователь, будут начальное число клеток и еды, количество еды, добавляемой каждый день, вероятность мутаций и уровень хищничества начальных клеток. Для ввода данных следует использовать ползунок, что предотвратит возможность ввода неверных данных. В качестве ответа на введенные данные пользователь сможет пронаблюдать за поведением организмов в течение длительного времени.

Для получения результатов симуляции, близких к реальным, следует вывести формулы для расчета энергии, затраченной организмом на продолжение жизнедеятельности, энергии, необходимой для размножения, а также получаемой из различных типов пищи - мяса и овощей.

Необходимо не допустить чрезмерной нагрузки на компьютер пользователя из-за большого количества клеток. Для этого нужно запретить функцию размножения при количестве клеток превышающем 60.

Назначение приложения - наглядная демонстрация процесса эволюции простейших организмов для учащихся школ на уроках биологии.

В первом разделе были проанализированы аналогичные программы и сформулированы основные задачи приложения.

2. Разработка приложения

2.1. Описание средств разработки

Так как приложение нужно разработать в формате 3D, удобнее всего будет использовать среду разработки Unity. Unity — это платформа разработки 3D-контента реального времени, которая предоставляет инструменты для разработки игр и охвата максимально широкого спектра устройств. Данная платформа имеет удобную визуальную среду разработки, что значительно упрощает процесс создания приложения. Другим важным преимуществом является поддержка физики твердых тел и встроенный инструмент генерации NavMesh, представляющий сцену в контексте, удобном для поиска оптимального пути к цели. Это позволит клеткам эффективнее искать еду и уходить от других клеток. Скрипты Unity написаны на языке C#.

Для написания скриптов используется среда разработки Microsoft Visual Studio. Это интегрированная среда разработки от Microsoft, основной инструмент разработки приложений для платформы .NET и Windows.

2.2. Выбор примитивов для отображения элементов системы

В среде разработки Unity возможно работать с моделями любой формы, созданными в сторонних программах для моделирования, однако существуют и примитивы - готовые трехмерные объекты, такие как сфера, куб, капсула, цилиндр или плоскость. Объекты можно растягивать относительно какой-либо оси. Так, например, из куба можно получить параллелепипед, а из сферы - капсулу. К этим игровым объектам также можно добавить физику твердых тел, что упростит моделирование их взаимодействия между собой.

Для создания имитационной модели необходимо выбрать используемые примитивы, которые будут отражать свойства элемента системы, за который они отвечают - клетки, еды или плоскости, на которой они будут находиться. При этом необходимо, чтобы клетка не могла выйти за пределы плоскости.

Соответственно, необходимо создать контейнер из плоскости и стен вокруг нее. Так как в настоящее время большинство экранов имеют прямоугольную форму, форма контейнера так же будет прямоугольной. В качестве плоскости и стен возьмем кубы, растянутые вдоль одной из координат. Так, например, левая стена будет являться кубом, растянутым в 10 раз по шкале x и в 5 раз по шкале y.

Формы и размер одноклеточных существ крайне разнообразны. Некоторые простейшие, например, амёба, даже не имеют постоянной формы[10].

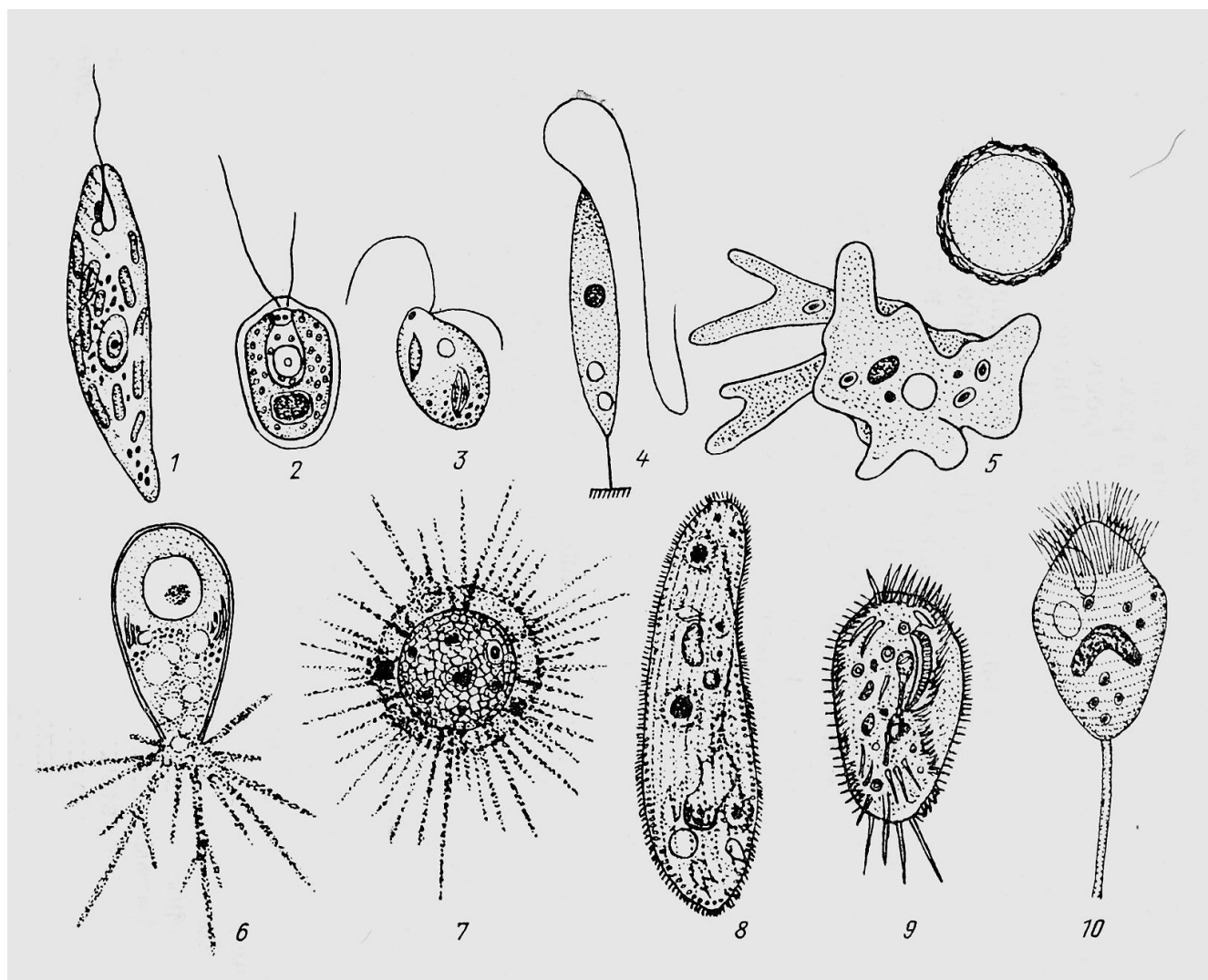


Рис.4.Формы простейших

Для отображения одноклеточного существа используем сферу. Это значительно упростит расчет затрат энергии клетки и моделирование движения, так как сфера, в отличие от капсулы, имеет только один радиус. Соответственно, для

отображения размера клетки нужно использовать лишь одну переменную. В качестве органа чувств сферы возьмем сферу большего радиуса.

Так как в качестве отображения пищи необходимо задать примитив, который будет значительно отличаться от клетки, для отображения растительной пищи используем куб зеленого цвета, мяса - красного.

2.3. Разработка структуры приложения

В приложении должно быть два окна - главное меню и основное окно симуляции. В главном меню пользователь имеет возможность задать настройки симуляции и запустить ее. В основной сцене должна воспроизводиться симуляция. Соответственно, приложение состоит из двух сцен. Сцена MainMenu запускается первой при открытии приложения и отвечает за окно главного меню, сцена Simulation запускается из первой сцены с помощью кнопки "Запуск" и отвечает за основное окно симуляции.

В главном меню пользователь имеет возможность с помощью ползунков задать настройки симуляции, отображающейся в основном окне. Передача информации о настройках между сценами происходит через объект gc сцены MainMenu, который получает информацию от ползунков и сохраняет ее во внутренних переменных и объект GameController сцены Simulation. Диаграмма сцен отображена на рис.5.

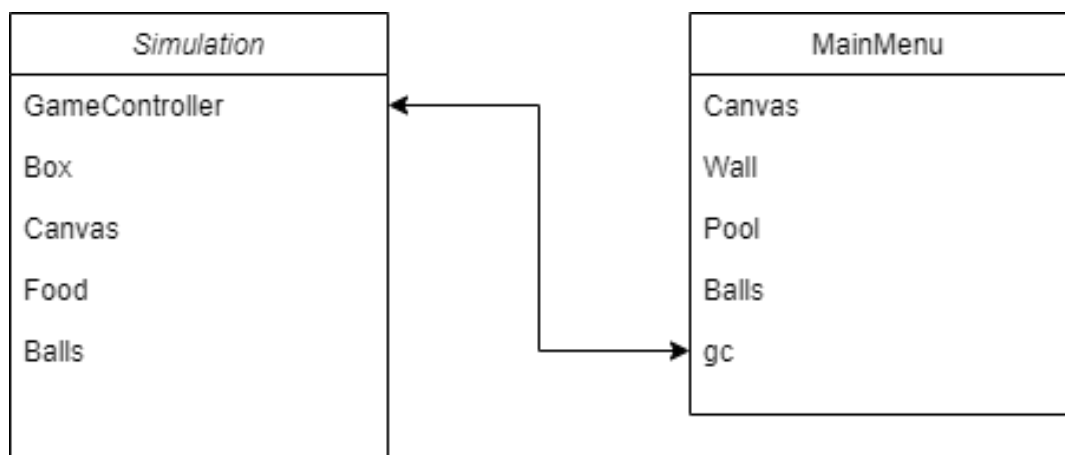


Рис. 5. Диаграмма сцен

Созданные скрипты и игровые объекты главного меню отражены на рис.6. Скрипты, присоединенные к одноименному объекту помечены как "script".

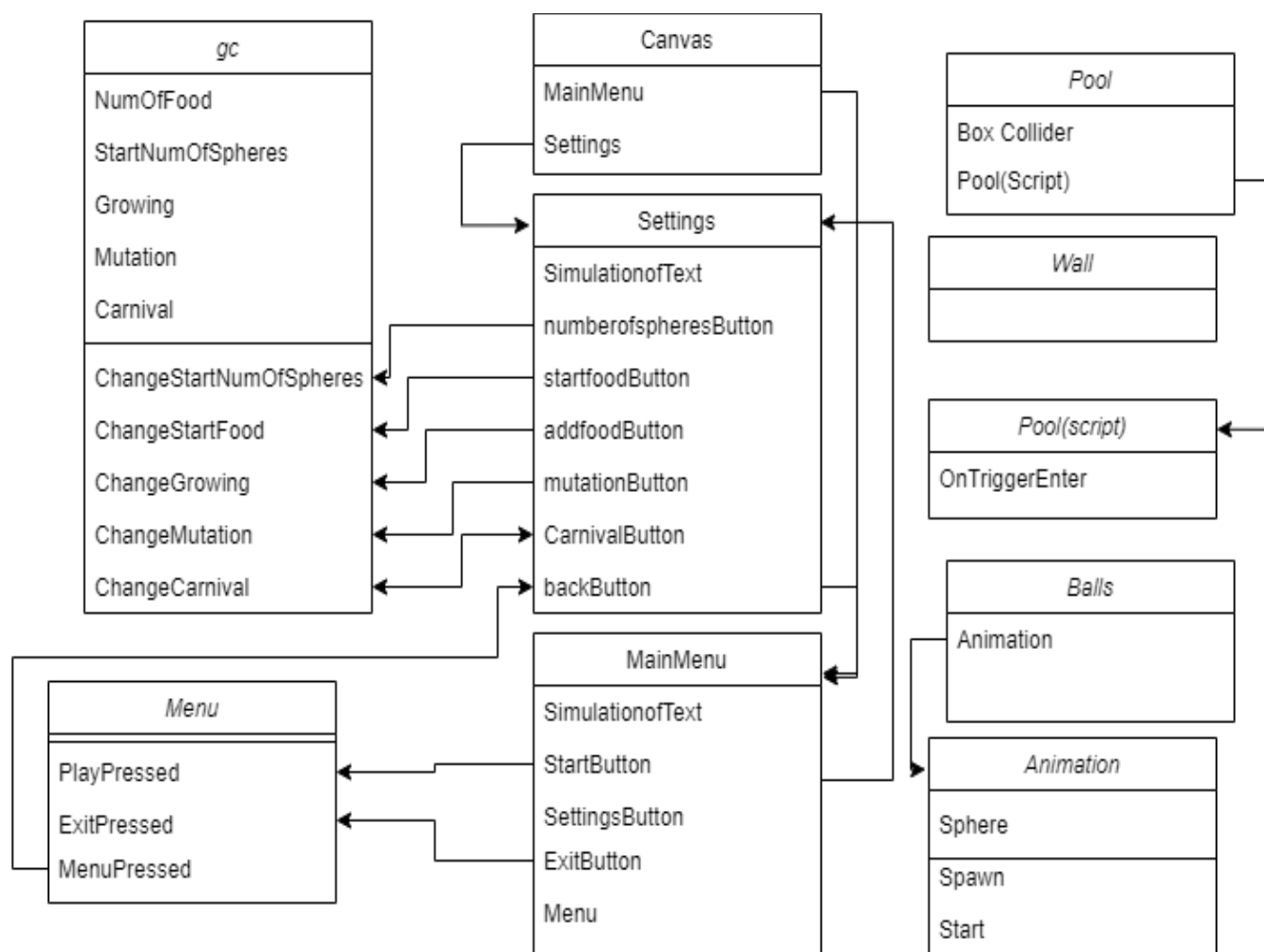


Рис.6. Диаграмма классов сцены главного меню

Методы и поля скриптов описаны в табл. 2.

Табл. 2

Описание полей и методов классов сцены главного меню

Имя процедуры	Назначение
Класс "Pool"(скрипт Unity) - класс поверхности, уничтожающей прикоснувшиеся объекты.	
OnTriggerEnter	Уничтожает прикоснувшийся объект
Класс "Animation"(скрипт Unity) - класс создания сфер случайного цвета	
Start	Вызывает функцию Spawn раз в 0,2 секунды
Spawn	Создает сферу случайного цвета
Класс "gc"(скрипт Unity) - класс, получающий информацию от ползунков, передвигаемых пользователем, и передающий ее в сцену Simulation	

Продолжение табл. 2

Имя процедуры	Назначение
ChangeStartNumOfSpheres	Получает информацию о количестве начальных сфер, заданном пользователем и сохраняет ее в соответствующей переменной
ChangeStartFood	Получает информацию о количестве начальной еды, заданном пользователем и сохраняет ее в соответствующей переменной
ChangeGrowing	Получает информацию о количестве еды, добавляемой каждый день, заданном пользователем и сохраняет ее в соответствующей переменной
ChangeMutation	Получает информацию о вероятности мутаций, заданной пользователем и сохраняет ее в соответствующей переменной
ChangeCarnival	Получает информацию об уровне хищничества начальных сфер, заданном пользователем и сохраняет ее в соответствующей переменной
Класс "Menu"(скрипт Unity) - класс, получающий информацию от кнопок, нажатых пользователем, и обрабатывающий ее	
PlayPressed	Запускает симуляцию
ExitPressed	Завершает программу
MenuPressed	Осуществляет переход из настроек в главное меню

Поля игровых объектов описаны в табл. 3.

Табл. 3

Описание полей игровых объектов сцены главного меню

Имя объекта	Назначение
MainMenu	- элемент пользовательского интерфейса, отвечающий за главное меню, относится к объекту Canvas
SimulationOfText	Надпись "Имитационная модель эволюции одноклеточных существ"
StartButton	Кнопка "Запуск"
SettingsButton	Кнопка "Настройки".
ExitButton	Кнопка "Выход"

Продолжение табл. 3

Имя объекта	Назначение
Menu	скрипт, описанный в табл.1.
Settings - элемент пользовательского интерфейса, отвечающий за настройки симуляции, относится к объекту Canvas	
SimulationOfText	Надпись "Имитационная модель эволюции одноклеточных существ"
NumberOfSpheresButton	Объект, содержащий в себе текст "Начальное число сфер" и ползунок, отвечающий за начальное число сфер
StartFoodButton	Объект, содержащий в себе текст "Начальное количество еды" и ползунок, отвечающий за начальное количество еды
AddFoodButton	Объект, содержащий в себе текст "Количество добавляемой еды" и ползунок, отвечающий за количество добавляемой еды
MutationButton	Объект, содержащий в себе текст "Вероятность мутаций" и ползунок, отвечающий за вероятность мутаций
CarnivalButton	Объект, содержащий в себе текст "Уровень хищничества" и ползунок, отвечающий за уровень хищничества начальных клеток
BackButton	Кнопка "Готово", возвращающая пользователя в главное меню
Pool - объект, уничтожающий другие объекты, которые прикасаются к нему	
Box Collider	Компонент, позволяющий отслеживать столкновения
Pool	Скрипт, описанный в табл.1.
Wall - объект, служащий фоном для главного меню	
Balls - пустой объект, к которому привязан скрипт Animation и к которому привязаны появляющиеся сферы	
Animation	Скрипт, описанный в табл.1.

Созданные скрипты и игровые объекты основной сцены отражены на рис.7. Скрипты, имеющие одно название с игровыми объектами помечены как "script".

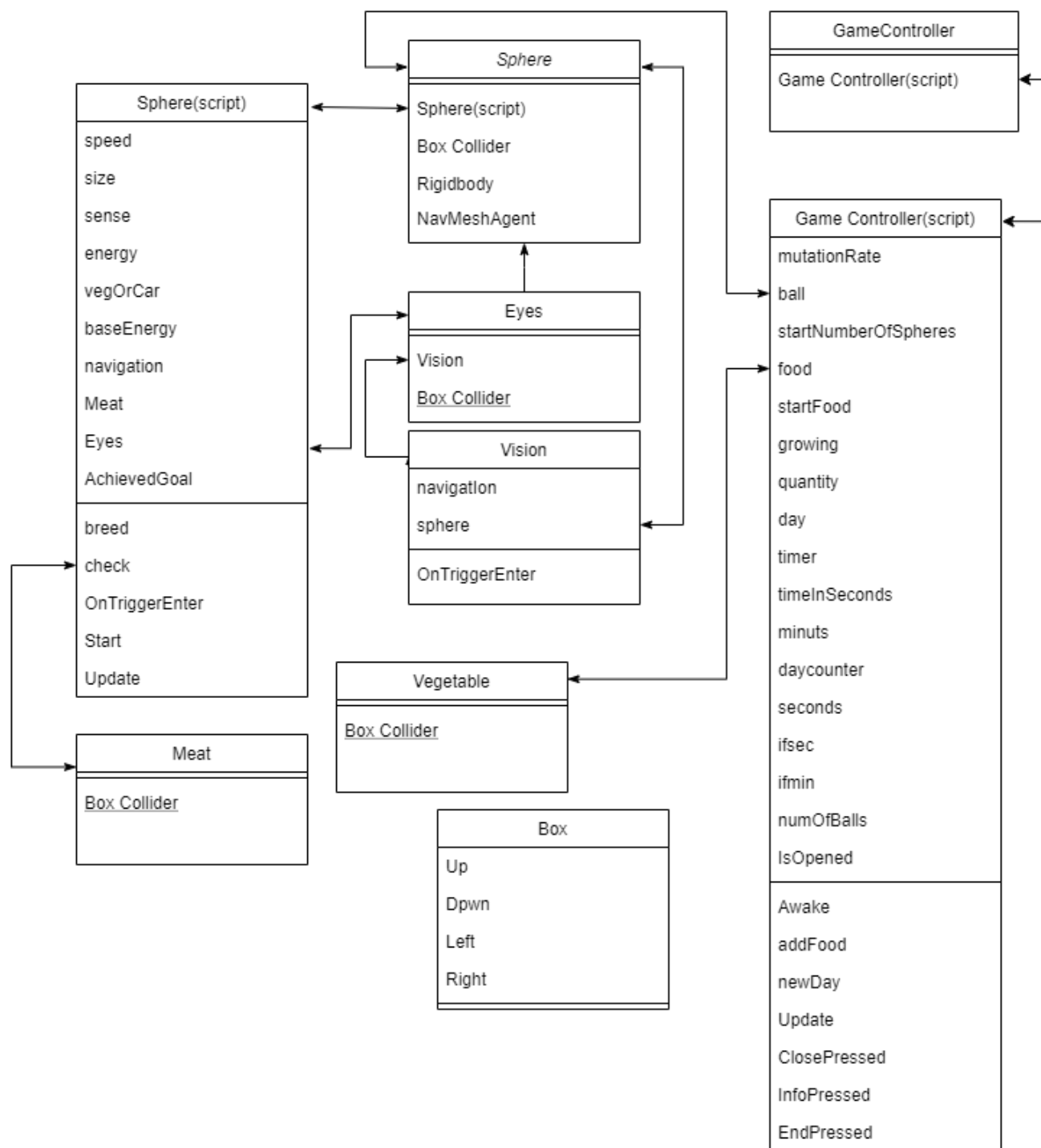


Рис.7. Диаграмма классов сцены симуляции

Методы и поля скриптов описаны в табл. 4.

Табл. 4

Описание полей и методов классов сцены симуляции

Имя процедуры	Назначение
Класс "Sphere"(скрипт Unity) - класс математической модели простейшего организма.	
breed	Создание дочернего объекта
check	Проверка количества энергии для выживания и размножения
OnTriggerEnter	Обрабатывает столкновение с едой или другой сферой
Start	Инициализация сферы
Update	Вызывает функцию движения с частотой смены кадров
Класс "Game Controller"(скрипт Unity) - класс управления основными компонентами модели	
AddFood	Добавляет овощи каждые 10 секунд
Start	Инициализация
newDay	Увеличивает и выводит номер текущего дня
Update	Выводит время и количество организмов на экране
ClosePressed	При нажатии клавиши "заккрыть" изменяет значение переменной IsOpened
Класс "Vision"(скрипт Unity) - класс области видимости сферы	
OnTriggerEnter	Обрабатывает столкновение с едой или другой сферой

Поля игровых объектов описаны в табл. 5.

Табл. 5

Описание полей игровых объектов

Имя объекта	Назначение
Sphere - игровой объект, отображающий одноклеточное существо	
"Sphere"(скрипт Unity)	Скрипт, описанный в табл.3.
Box Collider	Компонент, позволяющий отслеживать столкновения
RigidBody	Компонент, отвечающий за физику твердого тела
NavMeshAgent	Компонент, отвечающий за движение сферы в определенную точку
Eyes - игровой объект в виде сферы, отображающий зону видимости одноклеточного существа. Привязан к объекту Sphere	

Продолжение табл. 5

Имя объекта	Назначение
Vision	Скрипт, описанный в табл.3.
Box Collider	Компонент, позволяющий отслеживать столкновения
Vegetable - игровой объект в виде куба, отображающий растительную пищу	
Box Collider	Компонент, позволяющий отслеживать столкновения
Meat - игровой объект в виде куба, отображающий мясо	
Box Collider	Компонент, позволяющий отслеживать столкновения
GameController - пустой игровой объект	
GameController(script)	Скрипт, описанный в табл.3.
Box - игровой объект в виде куба, отображает дно коробки, в которой движутся сферы	
Up	Верхняя стенка коробки
Down	Нижняя стенка коробки
Left	Левая стенка коробки
Right	Правая стенка коробки

2.4. Расчет компонентов симуляции

Для получения результатов симуляции, близких к реальным, необходимо задать основные параметры для расчета затрачиваемой и получаемой энергии - базовую энергию, содержащуюся в пище, эффективность ее получения клеткой и ее затраты. Клетка получает энергию при взаимодействии с едой и тратит ее на поддержание своей жизнедеятельности и, если энергии достаточно, на размножение.

Основные параметры клетки - масса, скорость, чувствительность и уровень хищничества. Уровень хищничества должен влиять на получаемую энергию, а масса, скорость и чувствительность - на затрачиваемую. Для задания параметра базовой энергии, находящейся в пище, необходимо рассчитать количество энергии, затрачиваемой клеткой. Эта энергия в основном тратится на сохранение

клеткой своей массы, передвижение и восприятие предметов, находящихся вокруг.

Для сохранения массы, клетке необходимо получить равное ей количество энергии. Масса организма зависит от его размера. Так как в качестве отображения клетки принимается сфера, то зависимость массы от размера будем считать по формуле(1) для расчета массы сферы:

$$E_{\text{сохранения массы}} = m = \rho * V, \quad (1)$$

где ρ - плотность клетки, V - её объем, рассчитываемый по формуле:

$$V = \frac{4}{3} \pi * R^3$$

Тогда формула (1) принимает вид:

$$m = \rho * \frac{4}{3} \pi * R^3 \quad (2)$$

Энергия, потраченная на передвижение тела, равна работе по его перемещению, равной изменению его кинетической энергии и будет рассчитываться по формуле:

$$E_{\text{перемещения}} = \frac{1}{2} * m * v^2,$$

где m - масса организма, v - скорость.

Рассчитывая массу по формуле(2), получаем:

$$E_{\text{перемещения}} = \frac{1}{2} * \rho * \frac{4}{3} \pi * R^3 * v^2 \quad (3)$$

Так как глаза у организмов могут потреблять 5-8% от энергозатрат организма в покое, а общемозговые затраты достигают 23% [11], для расчета затрат простейших организмов возьмем аналогичное значение. Несмотря на отсутствие у одноклеточных полноценных глаз, у них есть ряд клеточных рецепторов, а также чувствительные молекулярные системы, такие, как, например, светочувствительный глазок у эвглены зеленой.

Выведем зависимость затрат энергии от радиуса органов чувствительности сферы $R_{\text{чувств.}}$. Для этого рассчитаем численное значение $E_{\text{затрачиваемая}}$.

Используя формулы (2) и (3), а также значение общемозговых затрат, получим:

$$\rho * \frac{4}{3} \pi * R^3 + \frac{1}{2} * \rho * \frac{4}{3} \pi * R^3 * v^2 = 0,77 * E_{\text{затрачиваемая}}$$

Следовательно,

$$E_{\text{затрачиваемая}} = \frac{\rho * \frac{4}{3} \pi * R^3 (1 + \frac{1}{2} * v^2)}{0,77} \quad (4)$$

Размер сферы может изменяться в пределах от 0,5 до 2. В качестве значения радиуса начальной сферы возьмем среднее значение, равное 1,25. Скорость может быть любым неотрицательным числом. Возьмем значение, равное 1. Радиусу области видимости присвоим значение 5.

Подставляя числовые значения, получим:

$$E_{\text{затрачиваемая}} = \frac{0,9982 * \frac{4}{3} \pi * 1,25^3 (1 + \frac{1}{2} * 1^2)}{0,77} = 15.9$$

Выведем коэффициент зависимости с учетом радиуса области видимости:

$$k = \frac{0.23 * 15.9}{5} = 0.73$$

В итоге получим формулу(5) расчета затрат энергии на поддержание жизнедеятельности:

$$E_{\text{затрачиваемая}} = \rho * \frac{4}{3} \pi * R^3 + \frac{1}{2} * \rho * \frac{4}{3} \pi * R^3 * v^2 + 0.73 * R_{\text{чувств}} \quad (5)$$

Энергия, получаемая сферой должна быть не меньше затрачиваемой для продолжения жизнедеятельности и значительно больше для деления. Для того, чтобы размножиться, сфере необходимо будет иметь количество энергии, равное сумме затрачиваемой на поддержание жизнедеятельности и на деление. Примем энергию, затрачиваемую на деление половине $E_{\text{затрачиваемая}}$. Тогда энергия, необходимая для размножения будет вычисляться по формуле:

$$E_{\text{размножения}} = 1,5 * (\rho * \frac{4}{3} \pi * R^3 + \frac{1}{2} * \rho * \frac{4}{3} \pi * R^3 * v^2 + 0.73 * R_{\text{чувств}}) \quad (6)$$

На энергию, получаемую организмом, будет влиять параметр хищничества, обозначаемый как vegOrCar . Чем больше этот параметр, тем эффективнее организмы переваривают мясо и тем хуже - овощи. Энергия, получаемая из овощей рассчитывается по формуле:

$$E_{\text{растения}} = \frac{(100 - \text{vegOrCar}) * E_{\text{базовая}}}{100}, \quad (7)$$

где $E_{\text{базовая}}$ - энергия одного овоща.

Так как для получения мяса нужно либо дождаться смерти клетки, либо убить клетку меньшего размера, быть хищником гораздо более рискованно, чем быть вегетерианцем. Поэтому энергия, получаемая из мяса должна быть больше. В среднем, калорийность мяса (в качестве аналогии бралось мясо кролика, курицы, индейки, говядина и свинина), - 170ккал, средняя калорийность овощей (для аналогии используются морковь, кабачки, огурец и брокколи) - 27ккал. Отсюда получается, что мясо примерно в 6 раз калорийнее овощей. Соответственно, энергия, получаемая из мяса, рассчитывается по следующей формуле:

$$E_{\text{мяса}} = 6 * \frac{\text{vegOrCar} * E_{\text{базовая}}}{100} \quad (8)$$

Предположим, для продолжения жизнедеятельности растительнойной (параметр хищничества равен 0) клетке с начальными параметрами необходимо съесть один овощ. Тогда, подставляя численное значение затрачиваемой энергии в формулу(7), получаем, что $E_{\text{базовая}}=159$.

2.5. Разработка алгоритмов

При запуске приложения пользователь видит главное меню и анимацию в нем - падающие сферы, которые уничтожаются при прикосновении к поверхности, находящейся вне поля зрения пользователя. Пока открыто окно главного меню, при помощи скрипта Animation создаются сферы, которые падают под действием физики Unity.

Алгоритм создания представлен на рис.8.

Для того, чтобы не перегружать компьютер пользователя большим количеством сфер, их необходимо удалять. В разрабатываемом приложении это происходит при помощи поверхности Pool, сделанной из куба, растянутого вдоль осей x и z, на которую падают сферы и привязанного к ней одноименного скрипта, алгоритм которого представлен на рис.9. Пока открыта сцена главного меню, при прикосновении какого-либо объекта поверхность уничтожает его.

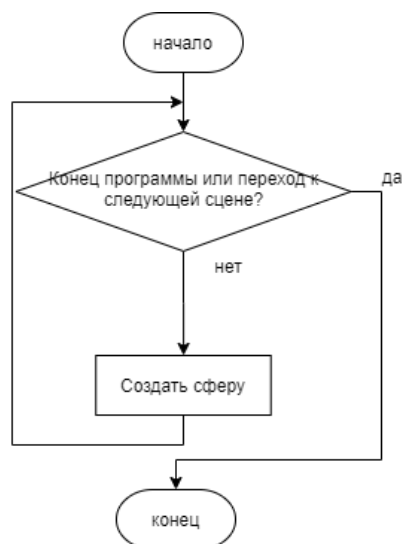


Рис.8. Алгоритм класса "Animation"

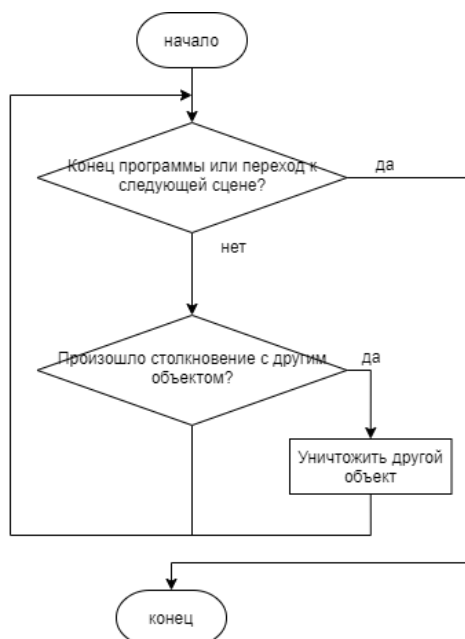


Рис.9. Алгоритм класса "Pool"

Нажатия пользователем одной из кнопок главного меню обрабатываются скриптом Menu, алгоритм которого представлен на рис.10.

При нажатии кнопки "Выход" происходит завершение программы. При нажатии кнопки "Настройки" кнопки главного меню скрываются, а ползунки настроек и кнопка возврата в главное меню становятся активными. Настройки пользователя обрабатываются и сохраняются скриптом gs. При нажатии кнопки "Запуск" происходит переход к сцене "Simulation" - запуск симуляции. Также в этом скрипте обрабатывается и нажатие кнопки "в главное меню" из сцены Simulation - открывается сцена MainMenu.

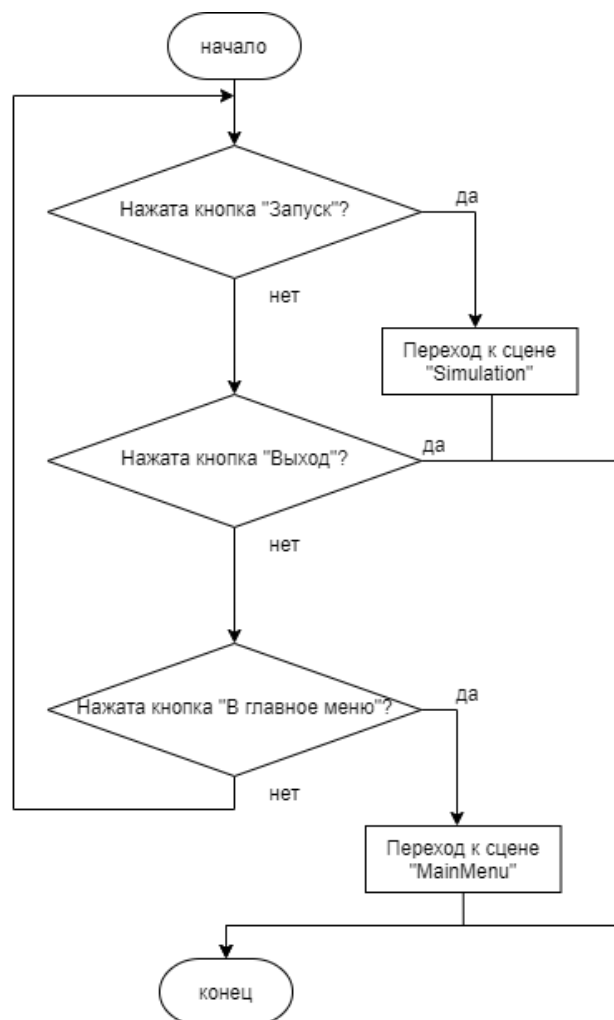


Рис.10. Алгоритм класса " Menu "

Сначала происходит выполнение процедуры Awake скрипта GameController. В ней задаются значения переменных startNumberOfSpheres, startFood,

MutationRate, growing и carnival. Это значения передаются с помощью объекта gc из сцены главного меню.

После выполнения Awake происходит запуск процедур Start() скриптов sphere и GameController. При запуске Start() скрипта sphere сфере задается ее цвет. Он зависит от уровня хищничества. В сферах-хищниках красная составляющая цвета имеет максимальное значение, а синяя и зеленая - нулевое. В сферах с нулевым уровнем хищничества максимальное значение имеет синяя составляющая, остальные - нулевое. После этого выполняется процедура InvokeRepeating, отвечающая за вызов с нулевой секунды каждые 10 секунд процедуры проверки check(), алгоритм которой представлен на рис.16. Затем в переменной goal запоминаются компоненты органа зрения и компоненту NavMesh, отвечающего за передвижения сферы задается скорость. Схема алгоритма представлена на рис.11.

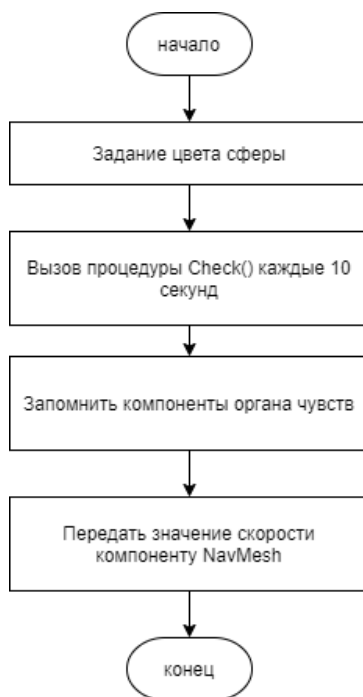


Рис.11. Алгоритм процедуры Start() скрипта sphere

При запуске Start() скрипта GameController происходит вызов процедуры newDay() с 0 секунды каждые 10 секунд и addFood() с 10 секунды каждые 10 секунд. NewDay() увеличивает время текущего дня и выводит его на экран, а addFood() добавляет несколько раз (количество задается пользователем) в точки

со случайными координатами от (-40;-20) до (40;20) объект Vegetable и делает его потомком пустого игрового объекта FOOD.

После этого добавляются первые сферы, количество которых задается пользователем. При этом они становятся потомками пустого игрового объекта BALLS и им задается параметр хищничества, заданный игроком в сцене главного меню. Затем добавляется количество еды, заданное игроком. Так же, как и в процедуре addFood() выбирается случайная координата от (-40;-20) до (40;20), в которую добавляется объект Vegetable, который становится потомком FOOD.

После выполнения Start() обоих скриптов происходит параллельное выполнение нескольких процедур - OnTriggerEnter(Collider other) скриптов sphere и vision, Update() скриптов sphere и GameController, вызовы check(), addFood() и newDay() с помощью InvokeRepeating, отслеживание и обработка нажатий на кнопки "заккрыть", "вывести информацию" и "завершить симуляцию". Общий алгоритм сцены Simulation представлен на рис.12 и рис.13.

Процедура OnTriggerEnter(Collider other) скрипта Vision отвечает за область чувствительности сферы и обрабатывает ее пересечения другими объектами в случае, если значение переменной AchieveGoal=true. Если сфера eyes столкнулась с другой сферой, радиус которой в 1,5 больше, чем радиус родительской сферы, то значению переменной navigation, отвечающей за точку назначения сферы задается значение, обратное значению координат другой сферы. В ином случае, если другая сфера в 1,5 раза меньше родительской и параметр хищничество родительской сферы больше 50, она принимается за еду и ее координаты становятся точкой назначения клетки.

При попадании в зону видимости мяса (при уровне хищничества большем, чем ноль) или овоща (при уровне хищничества меньшем, чем 100) вектору navigation присваивается значение координат данного объекта. При этом переменной AchievedGoal задается значение false. Алгоритм обработки представлен на рис.13.

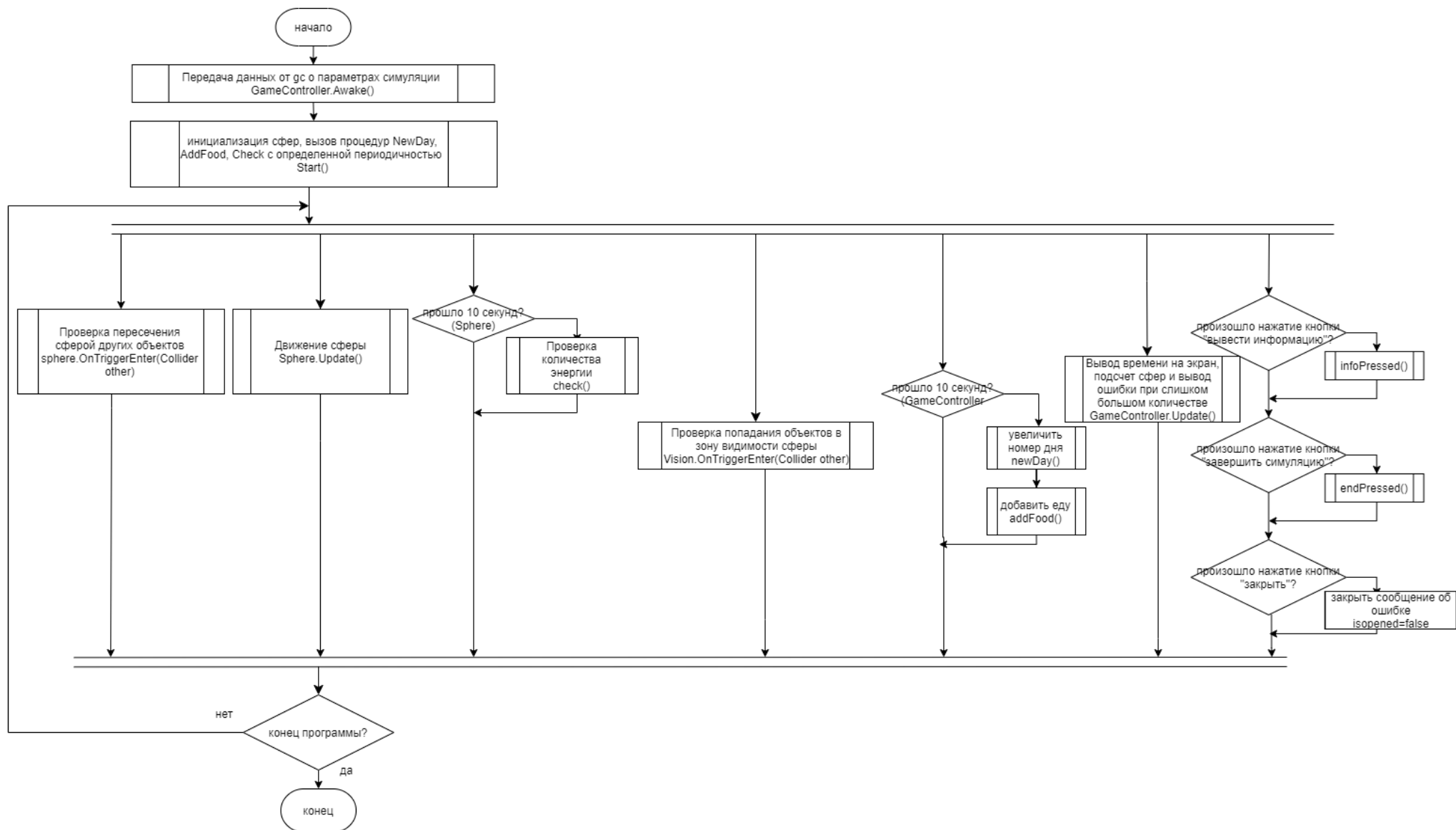


Рис.12. Общий алгоритм сцены Simulation

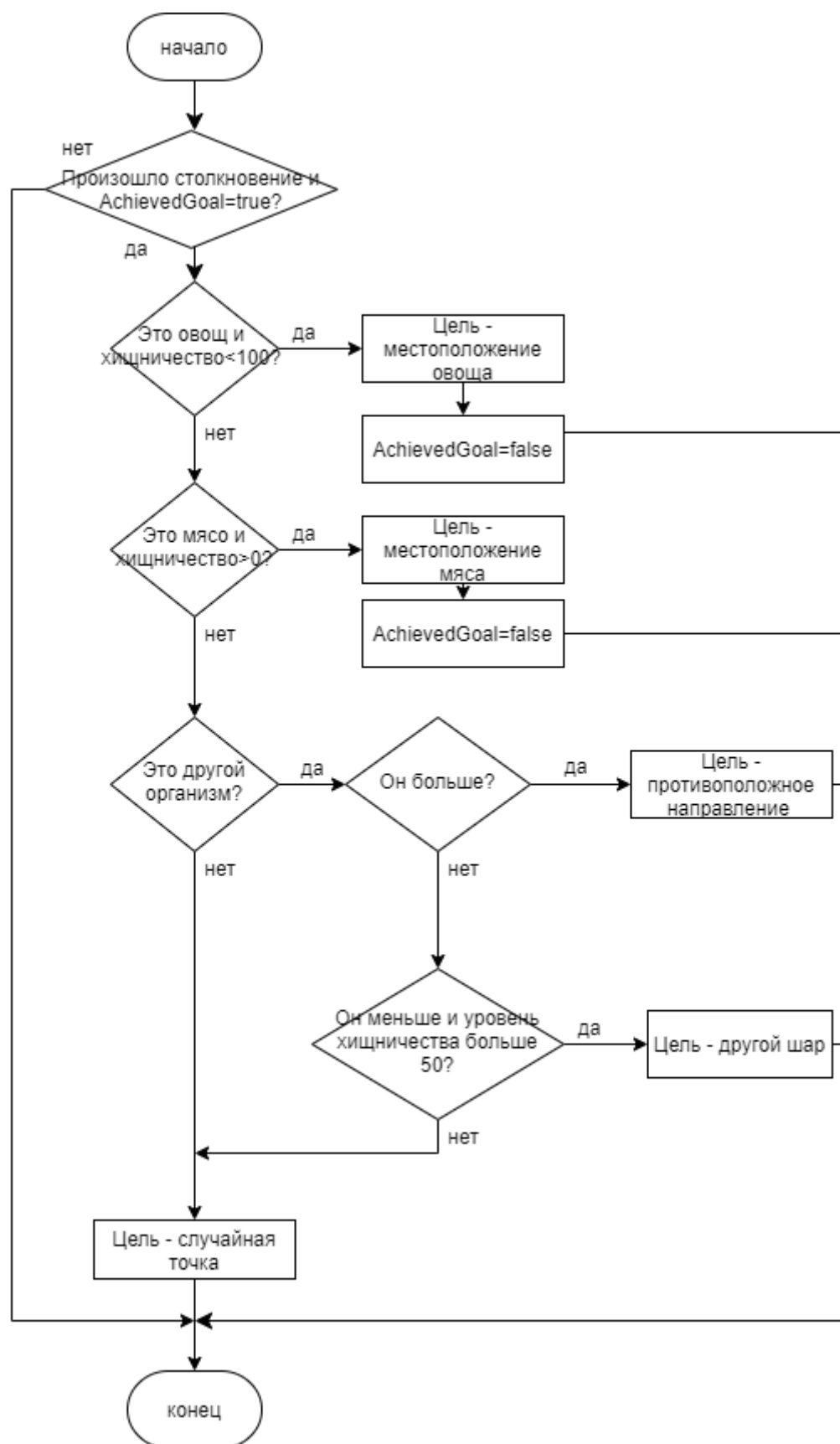


Рис.13. Алгоритм обработки попадания предмета в область видимости

Процедура `OnTriggerEnter(Collider other)` скрипта `Sphere` отвечает за обработку столкновений сферы с другими объектами.

Если сфера столкнулась с другой сферой с размером, меньшим в 1,5 раза, то она забирает энергию другой сферы и уничтожает ее. При пересечении овоща, сфера получает энергию, рассчитываемую по формуле(7), при столкновении с мясом - энергию, рассчитываемую по формуле(8), после чего переменной `AchievedGoal` присваивается значение `true`.

В процедуре `Update()` скрипта `sphere`, вызываемой каждую смену кадра, сфера получает цель от органа чувств и передает ее компоненту `NavMesh`. При этом, если данная цель достигнута(координаты цели совпадают с координатами сферы), сфере задается другая точка назначения.

Процедура `Update()` скрипта `GameController` отвечает за подсчет количества сфер и вывод времени на экран. При достижении 60 клеток, на экран выводится подсказка, предупреждающая пользователя о том, что при данном количестве сферы размножаться не будут, так как это может привести к замедлению работы компьютера. В случае, если пользователь уже закрывал эту подсказку, она не будет выводиться, за что отвечает переменная `isopened`. При запуске приложения ее значение `true`, при нажатии кнопки `close` вызывается процедура, изменяющая ее значение на противоположное. Время подсчитывается и выводится по алгоритму, изображенному на рис.14.

Каждые 10 секунд происходит вызов процедур `check()`, `addFood()` и `newDay()`.

Алгоритм `check()` изображен на рис.15. Сначала из энергии сферы вычитается значение, рассчитанное по формуле(5). В случае, если энергия сферы оказывается отрицательной, сфера уничтожается, а на ее место добавляется игровой объект `Meat`. В ином случае, если у сферы осталось количество энергии большее, чем необходимо для размножения (рассчитывается по формуле(6)), ее значение уменьшается и происходит вызов процедуры `breed()`, алгоритм которой представлен на рис.16.

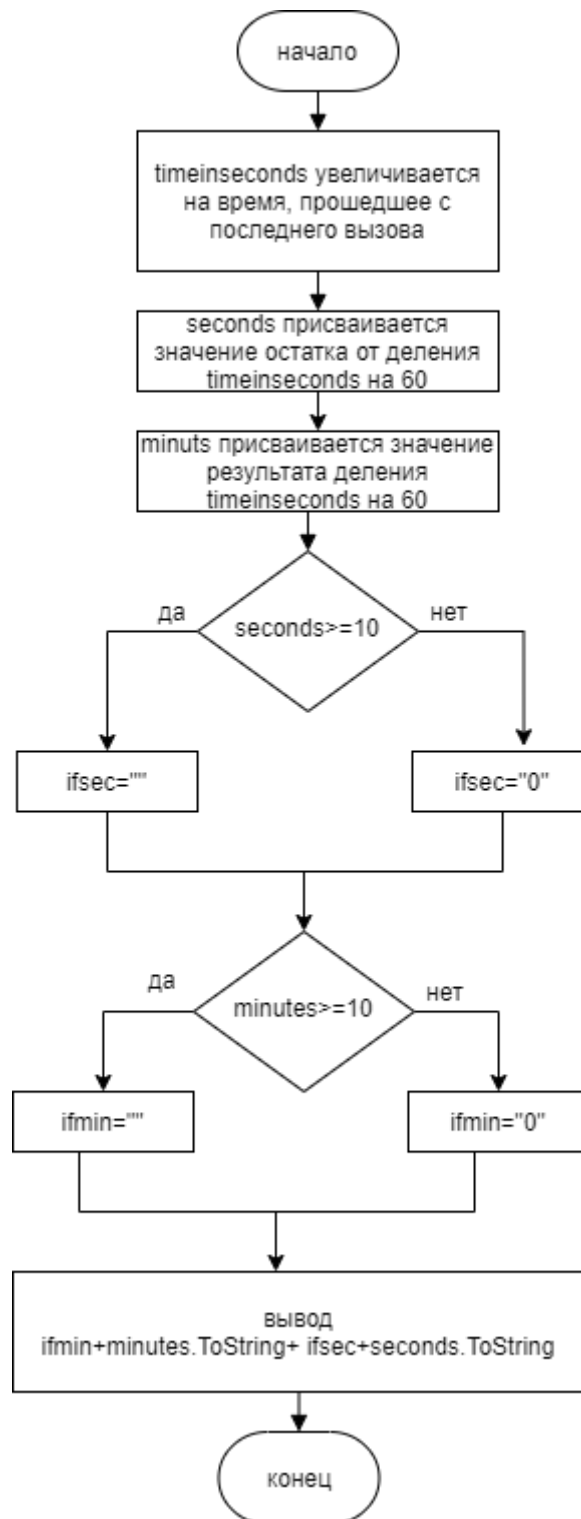


Рис.14. Алгоритм вывода времени на экран

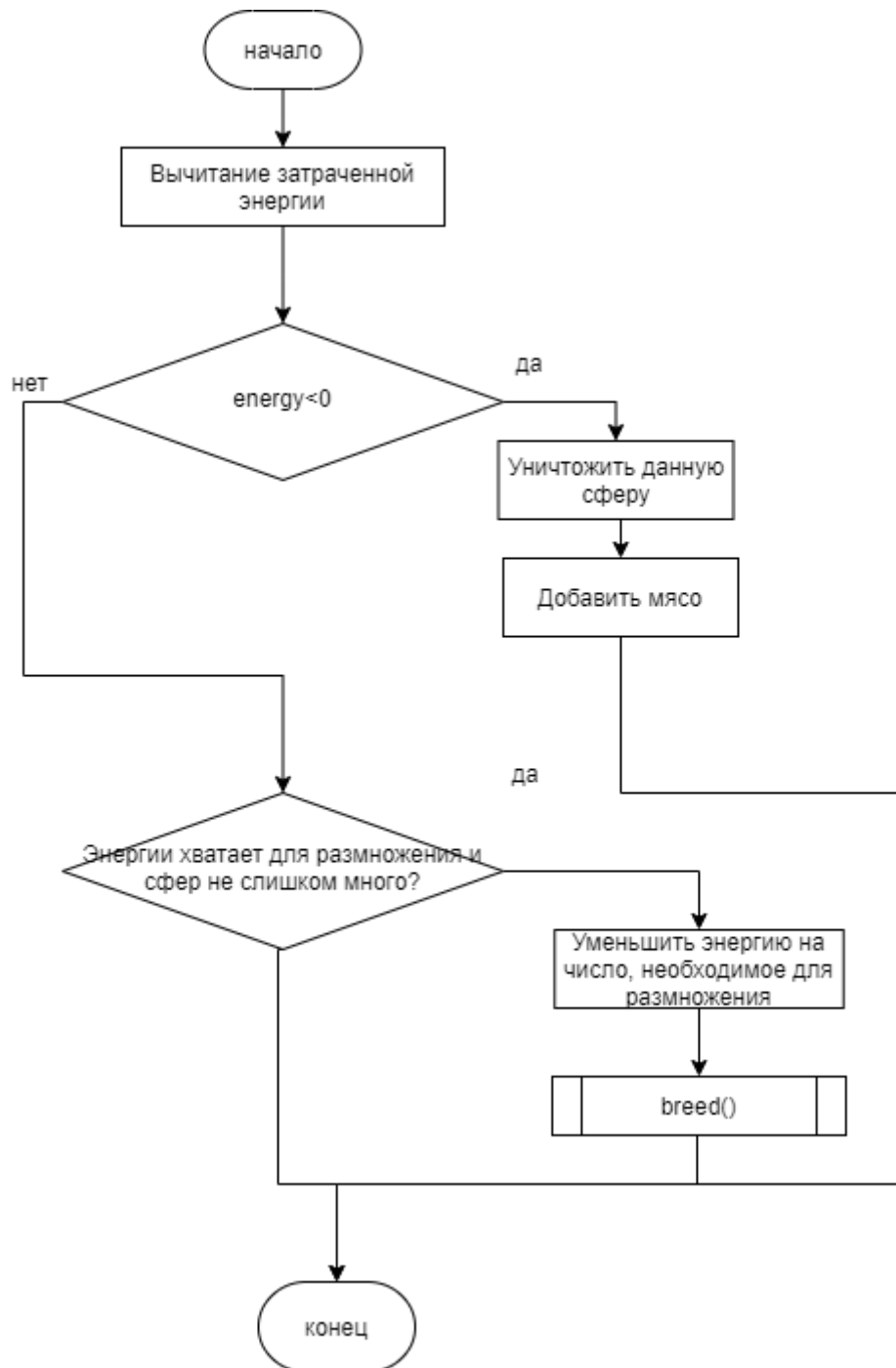


Рис.15. Алгоритм процедуры `check()`

При ее вызове создается копия родительской сферы. Параметры новой сферы запоминаются в отдельной переменной, так как многочисленное обращение к ним будет замедлять работу приложения. После этого задается значение энергии сферы и переменной `AchievedGoal`. Затем задается одно четырехзначное случайное число от 0 до 9999, каждая цифра которого отвечает за мутацию отдельного параметра сферы.

Если первая цифра получилась меньше, чем значение, заданное пользователем, происходит изменение скорости в пределах $(-0,5;0,5)$. В случае, если скорость получилась отрицательной, ее значение приравнивается к 0. Аналогичные ограничения имеет и мутация области видимости, за которую отвечает вторая цифра случайного числа. От значения третьей цифры числа зависит, произойдет ли изменение уровня хищничества в пределах $(-5;5)$. Уровень хищничества не может быть больше 100 или меньше 0. Четвертая цифра отвечает за изменение радиуса сферы в пределах $(-0,5;0,5)$. Если получившееся значение больше, чем 2, оно приравнивается к 2, в ином случае, если радиус сферы меньше 0,5, ему присваивается значение 0,5.

После всех мутаций сфере задается физический размер и она становится потомком игрового объекта BALLS.

Параллельно остальным процедурам происходит также ожидание и обработка нажатий на кнопки "Информация о системе", "завершить симуляцию" и "в главное меню". Нажатие последней обрабатывается скриптом Menu.

При нажатии на кнопку "Информация о системе" происходит вызов процедуры InfoPressed() алгоритм, которой изображен на рис.17.

При этом создаются несколько переменных для средних значений скорости, размера, энергии, уровня хищничества и радиуса обзора. После этого с помощью процедуры GameObject.FindGameObjectsWithTag("Ball") создается массив из сфер. Затем с помощью цикла for происходит обход массива и прибавление всех компонентов каждой сферы к соответствующей переменной. После этого каждая переменная делится на длину массива, а в переменную NumOfFood записывается количество объектов с тегами "мясо" и "овощи".

Так как в этой процедуре происходит три обращения к GameObject.FindGameObjectsWithTag(), опрашивающей все объекты, принадлежащие запущенной сцене, ее нужно вызывать только по запросу пользователя, в ином случае эта процедура сильно замедлила бы работу приложения.

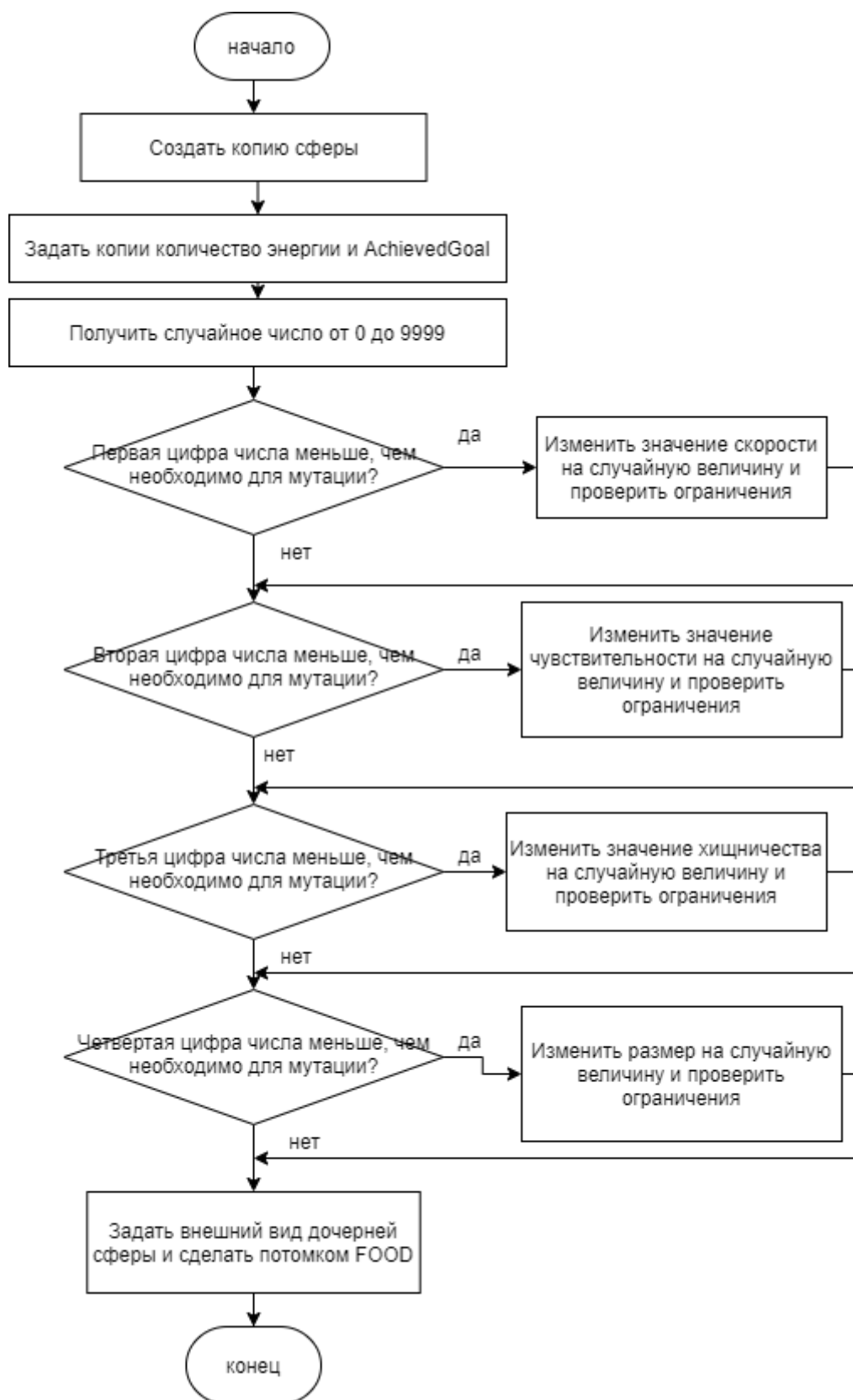


Рис. 16. Алгоритм размножения сферы

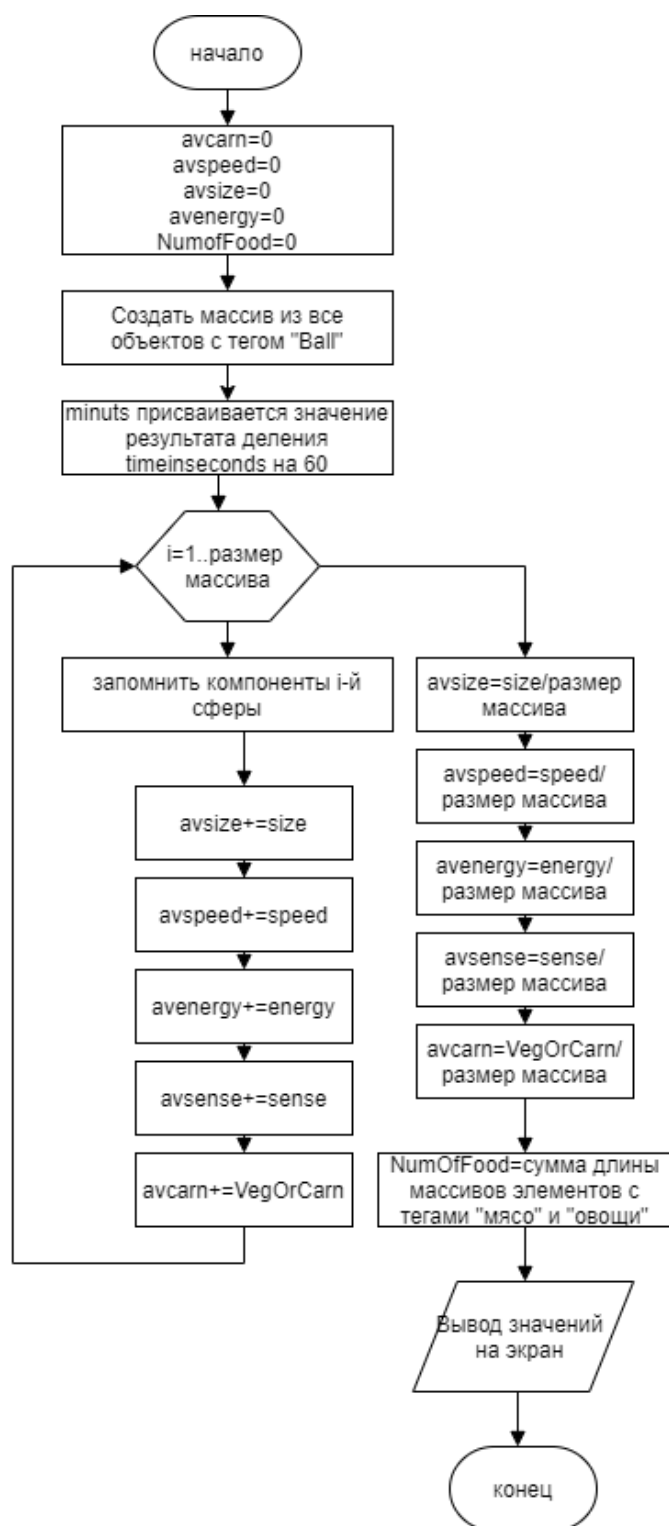


Рис.17.Алгоритм подсчета и вывода информации на экран

При нажатии кнопки "Завершить симуляцию" в процедуре EndPressed() так же, как и при вызове InfoPressed() подсчитываются и выводятся средние значения параметров клетки. Кроме этого, становятся неактивными объекты, содержащие в себе информацию о дне и времени,, прошедшем с момента запуска приложения,

кнопка "Информация о системе", пустые игровые объекты FOOD и BALLS, а также все их потомки. Таким образом, симуляция останавливается.

Так как приложение создается в образовательных целях, происходит анализ информации. В случае, если средний уровень хищничества больше 50, выводится надпись "Большая часть сфер оказалась хищниками. Возможные причины: а) В данных условиях не оказалось достаточного количества растительной пищи и клетки были вынуждены учиться питаться мясом б) В данных условиях было много маленьких клеток и большие клетки научились их есть", в противном случае выводится "В данных условиях было достаточно растительной пищи, поэтому большая часть сфер оказалась вегетерианацами".

2.4. Разработка пользовательского интерфейса

Имитационная модель разработана для учащихся средней школы и учителей биологии. Каждому пользователю будут доступны все окна приложения.

При запуске приложения открывается окно настроек Unity, изображенное на рис.18. В нем можно выбрать настройки качества изображения, разрешение экрана и экран (при наличии нескольких экранов). Также можно установить полноэкранный режим.

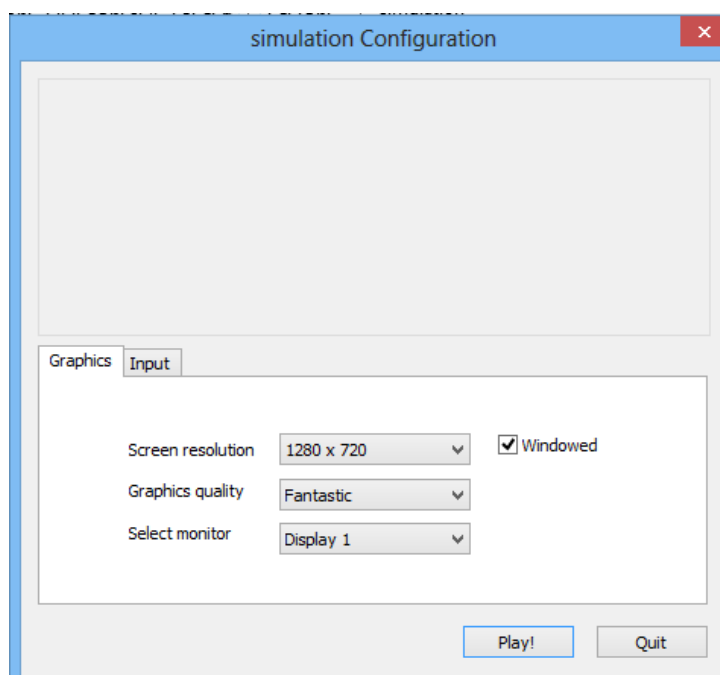


Рис.18. Окно настроек Unity

После нажатия кнопки "Play!" появляется главное меню приложения. Оно представлено на рис.19.

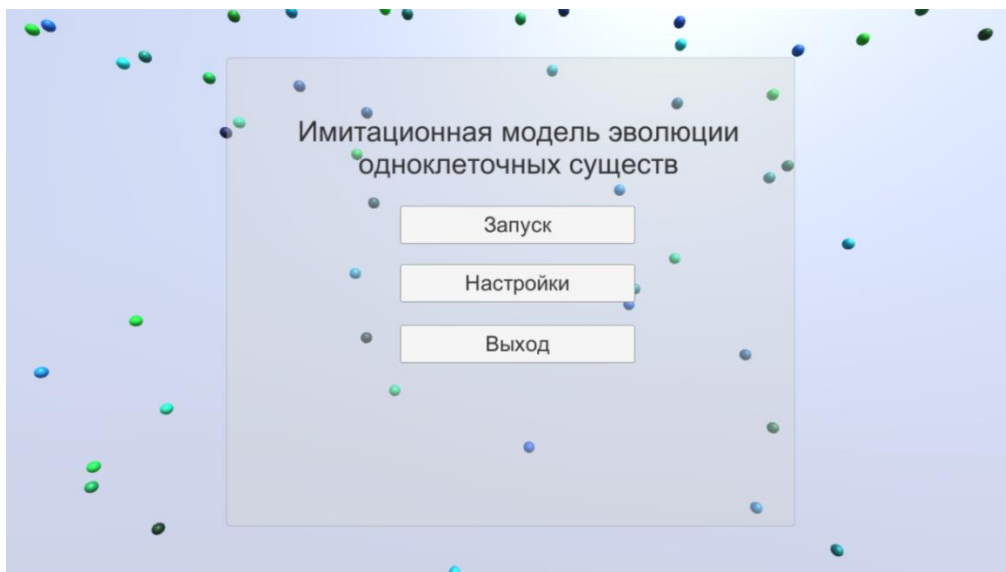


Рис.19. Главное меню

Пользователь может запустить нажатием на кнопку "Запуск" симуляцию сразу или настроить ее параметры самостоятельно. Окно настроек, изображенное на рис.20, открывается при нажатии кнопки "Настройки". В окне находятся ползунки, с помощью которых можно выбрать начальное количество сфер и еды, количество добавляемой еды и вероятность мутаций. При нажатии на кнопку "Выход" происходит завершение работы приложения.

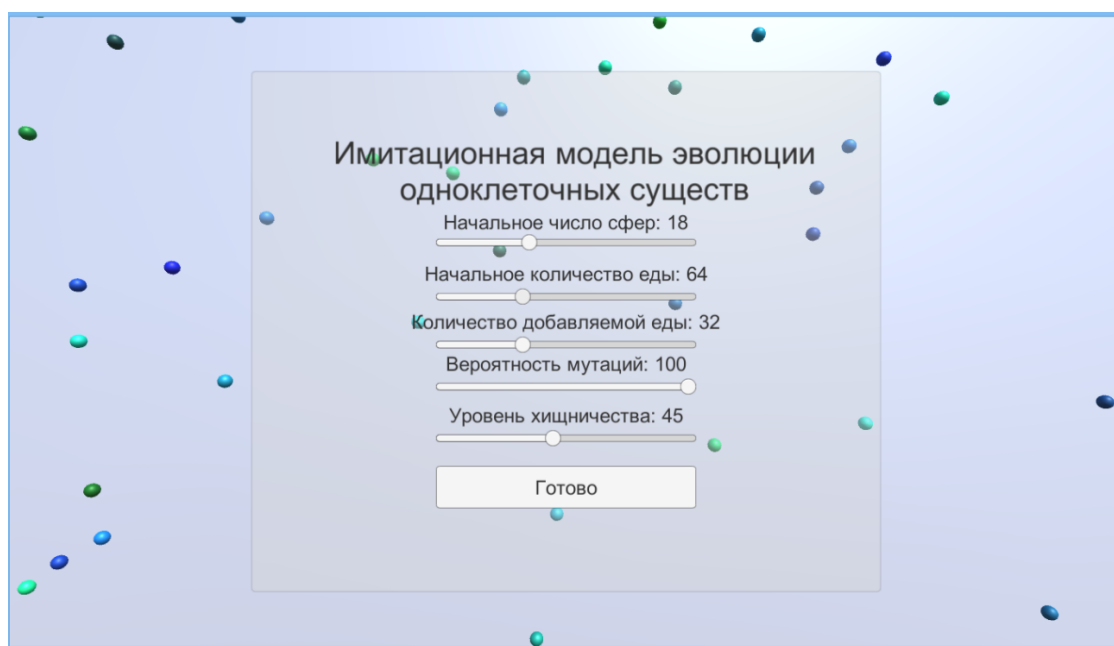


Рис. 20. Окно настроек

После ввода необходимых данных пользователь должен нажать кнопку "Готово", а затем кнопку "Запуск". После нажатия запускается имитационная модель, окно которой изображено на рис.21. В её окне выводится информация о номере текущего дня и времени, прошедшем с момента запуска, а также присутствуют кнопки "информация о системе", "завершить симуляцию" и "в главное меню".

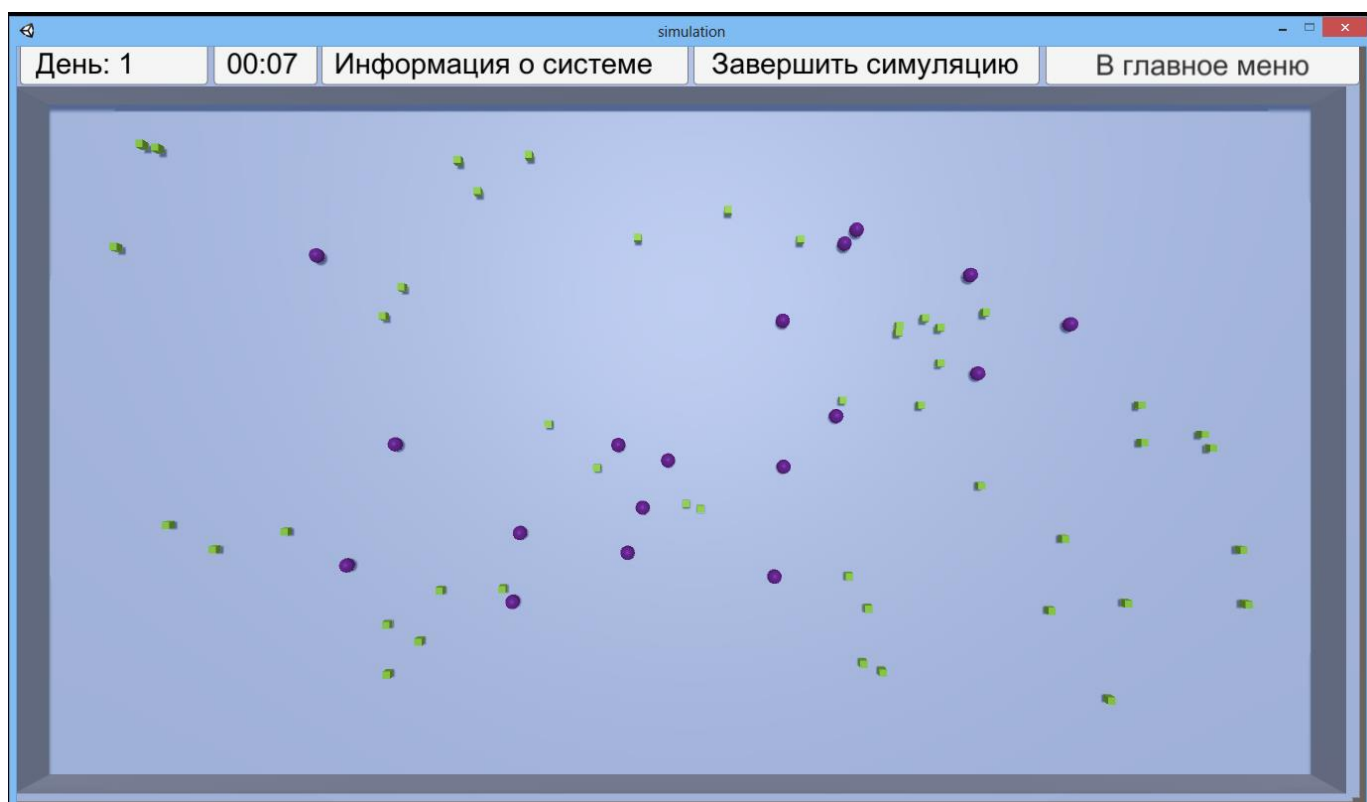


Рис. 21. Основное окно программы

При нажатии на кнопку "информация о системе" на экран выводится информация о числе клеток, их средних параметрах, а также об общем количестве еды на экране. Пользователь может закрыть окно информации или обновить его значения с помощью соответствующих кнопок. Информация представлена на рис.22.

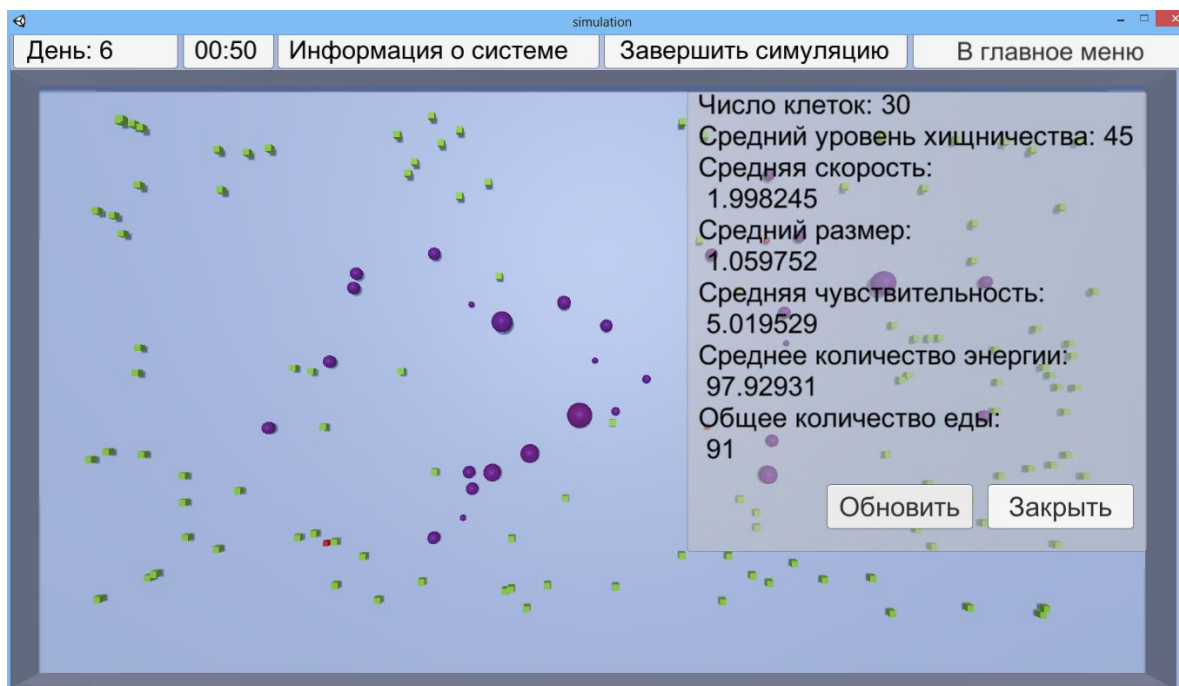


Рис.22. Информация о системе

При нажатии кнопки "Завершить симуляцию" на экран выводится информация о количестве сфер на момент окончания, их средних параметрах, а также один из выводов о возможных причинах такого результата, что отображено на рис.23. Симуляция останавливается и пользователю становится доступна лишь кнопка "В главное меню" для того, чтобы выйти и запустить симуляцию заново или завершить работу приложения.

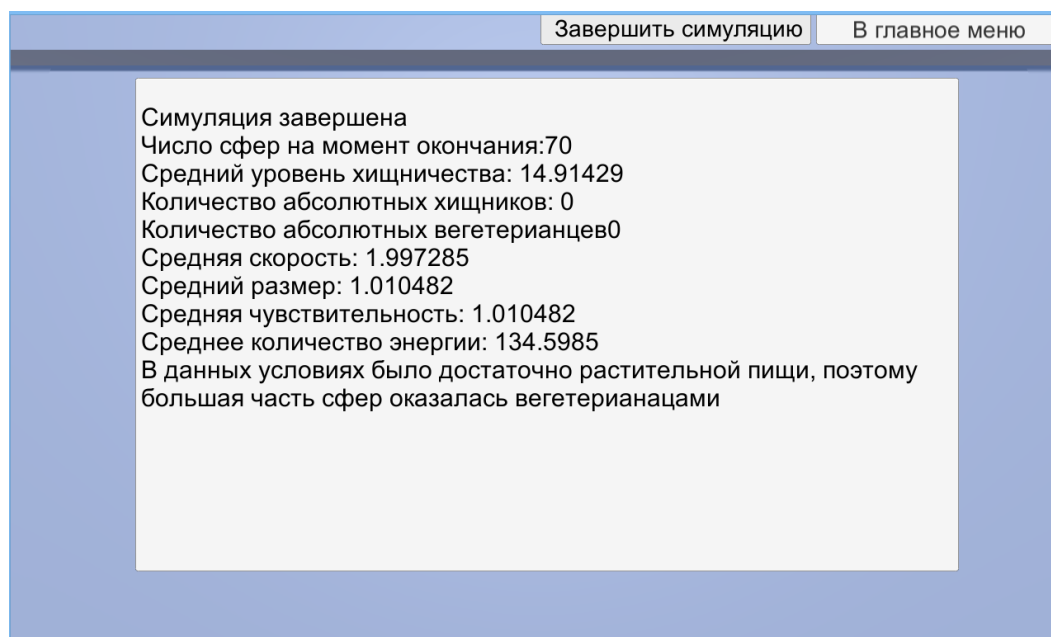


Рис.23. Окно завершения симуляции

При превышении максимального количества сфер выводится подсказка о том, что сферы не будут размножаться при данном количестве. Она изображена на рис.24. Пользователь может нажать на кнопку "заккрыть" и тогда эта подсказка больше не будет появляться.

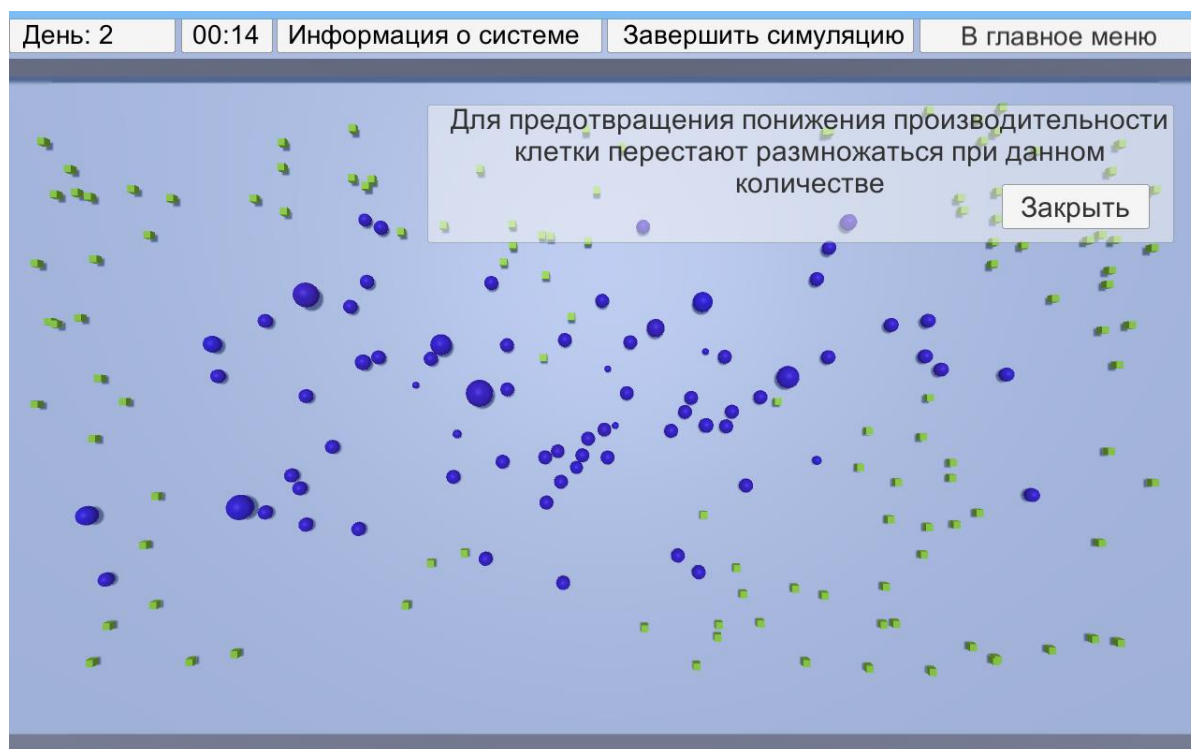


Рис.24. Подсказка о превышении допустимого числа сфер

Во втором разделе были описаны средства разработки, произведен вывод формул для расчета параметров симуляции, выбраны основные примитивы и разработаны диаграммы классов и основные алгоритмы.

3. Реализация и тестирование программы

3.1. Реализация программы

Процесс разработки приложения состоял из трех частей - создания сцены главного меню, сцены симуляции и написания скриптов для объектов этих сцен. Сцены создавались в среде разработки Unity, скрипты - в среде Visual Studio с использованием библиотеки UnityEngine. Для реализации скриптов использовались внутренние инструменты и процедуры Unity, такие, как NavMesh, Instantiate, Destroy или OnTriggerEnter.

Процедура Instantiate используется для создания объектов, являющихся клоном заданного объекта. В программе она используется для создания начальных сфер, их потомков и еды.

Процедура Destroy уничтожает заданный объект. Один из примеров ее использования - уничтожение еды сферой, прикоснувшейся к ней.

Для обработки столкновений сфер и определения пересечения зоны видимости сферы используется OnTriggerEnter. Это событие, которое происходит при столкновении двух игровых объектов. Оно обрабатывается в скрипте Vision и в скрипте Sphere. В первом случае происходит определение объекта, который попал в зону видимости сферы и выбор действия, которое зависит от объекта. Во втором - поглощение объекта, если он является едой или маленькой сферой.

Для расчета движения сферы использовался внутренний инструмент для генерации навигационных мешей NavMesh. Он позволяет сфере выбирать оптимальный путь для достижения цели с учетом препятствий. На рис.25. изображена сфера с целью в случайной точке и путь к ней.

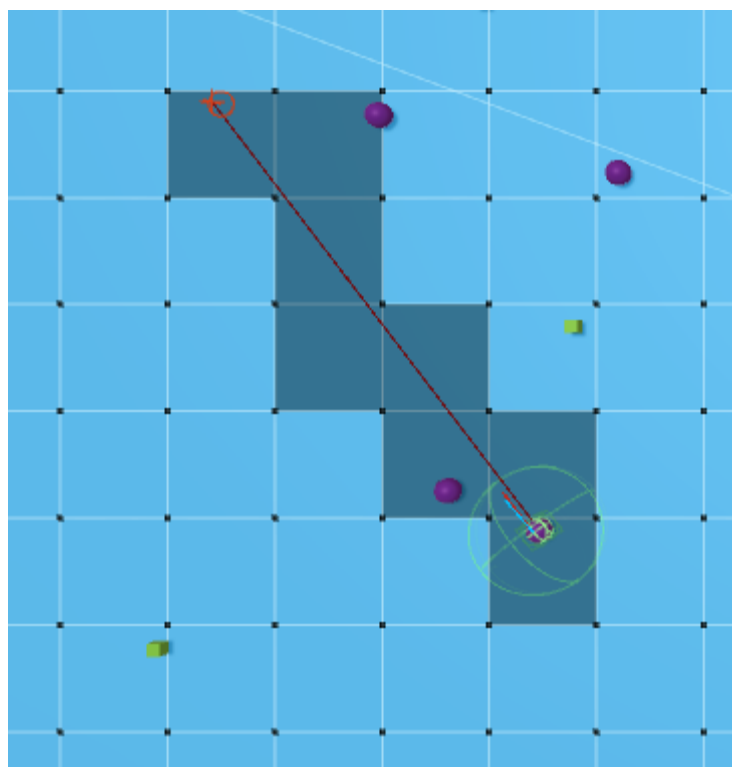


Рис.25. Построение пути к цели

Для использования NavMesh необходима поверхность, по которой будет передвигаться сфера и сама сфера, к которой добавлен компонент NavMesh Agent. У этого компонента есть следующие свойства, которые важны для расчета пути сферы:

- Радиус агента, используется для подсчета коллизий между агентами и препятствиями. Равен радиусу сферы.
- Скорость - максимальная скорость, которую может достичь агент. Равна скорости сферы.
- Дистанция остановки - расстояние до цели, на котором агент останавливается. Равна нулю.

Для того, чтобы найти путь в определенную точку, нужно сначала сопоставить начальные и конечные точки с их ближайшими полигонами. Затем начинается поиск пути с начального полигона, посещая всех соседей, пока не достигнут конечный полигон. Последовательность полигонов, описывающих путь от начала до конечного полигона называется коридором. Агент доберется до

места назначения, всегда направляясь к следующему видимому углу коридора[12].

Код одного из скриптов представлен в Приложении Б.

3.2. Тестирование

Основная часть тестирования программы заключается в проверке работоспособности на нескольких конфигурациях оборудования

Табл. 6

Конфигурации, используемые для тестирования

	Конфигурация 1	Конфигурация 2
Процессор	Intel(r) Core(TM) i5-3230M CPU @ 2.60GHz	Intel core i3
ОЗУ	8 Гб	6 Гб
ЗУ	500 Гб	512 Гб SSD 512 Гб HDD
Операционная система	Windows 8	Windows 10

Среда разработки Unity дает возможность настроить параметры для более слабых и сильных устройств. Это отображено на рис.18 - пользователь может выбрать параметры для запуска, подходящие его устройству.

Суть тестов на работоспособность заключается в запуске приложения с минимальным качеством изображения на различных конфигурациях оборудования. В результате данного тестирования не было выявлено ошибок.

Также необходимо произвести проверку правильности передачи данных из одной сцены в другую. На рис.26 отображены параметры, заданные в ходе тестирования. На рис.27 можно увидеть, что параметры симуляции переданы верно. В ходе тестирования была выявлена и исправлена ошибка, в результате которой данные передавались неверно при запуске сцены меню более одного раза.

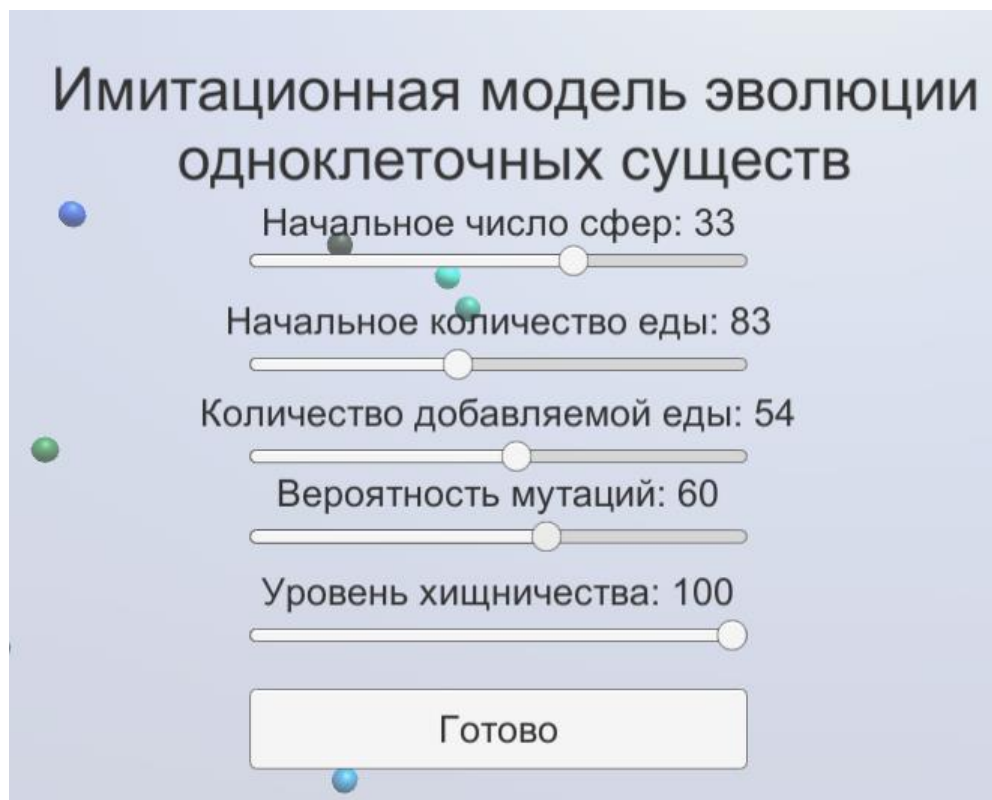


Рис.26. Заданные параметры для тестирования



Рис.27. Заданные параметры для тестирования

В третьем разделе была реализовано и протестировано приложение, реализующее имитационную модель эволюции простейших существ.

Заключение

В результате выполнения выпускной квалификационной работы было разработано приложение, реализующее имитационную модель эволюции простейших организмов. Было разработано две сцены, для которых написано 7 скриптов, регулирующих поведение объектов в этих сценах.

Приложение имеет все основные свойства моделируемой системы и может использоваться для экспериментов над ней. Разработанная имитационная модель позволяет получать результаты при различных заданных условиях, выраженных в количестве пищи, которую может добыть организм, вероятности мутаций и типа питания организма. Это позволяет пронаблюдать возможные пути эволюции при изменении условий обитания организма. Кроме того, приложение помогает пользователю проанализировать полученные в зависимости от условий.

Данная имитационная модель может применяться для в школах на лабораторных работах по биологии. Это поможет повысить интерес учащихся к учебе и их результативность.

Приложение разработано с возможностью улучшения. Возможные способы его доработки:

- Создание мобильной версии.
- Добавление большего числа подсказок для пользователя.
- Добавление новых параметров симуляции, влияющих на результат.

Список использованной литературы

1. Unity User Manual [Электронный ресурс] - Режим доступа: <https://docs.unity3d.com/ru/2018.4/Manual/CreatingScenes.html>
2. Мэннинг Д., Батфилд-Эддисон П. Unity для разработчика. Мобильные мультиплатформенные игры. - СПб.: Питер, 2018 - 304 с.
3. Unity User Manual [Электронный ресурс] - Режим доступа: <https://docs.unity3d.com/ru/2018.4/Manual/CreatingAndUsingScripts.html>
4. Википедия [Электронный ресурс] - Режим доступа: <https://ru.wikipedia.org/wiki/Мутация>
5. Википедия [Электронный ресурс] - Режим доступа: https://ru.wikipedia.org/wiki/Естественный_отбор
6. Национальный исследовательский институт генома человека [Электронный ресурс] - Режим доступа: <https://www.genome.gov/genetics-glossary/Genome?id=90>
7. Вавилова Л.Н., Панина Т.С. Современные способы активизации обучения. - М.: Издательский центр "Академия", 2006. - 176 с.
8. Алиев Т.И. Основы моделирования дискретных систем. – СПб:СПбГУ ИТМО, 2009. – 363 с.
9. Догель В. А., Полянский Ю. И., Хейсин Е. М. Общая протозоология. — М.: Изд-во АН СССР, 1962. — 592 с
10. Бабьева И.П, Звягинцев Д.Г., Зенова Г.М. Биология почв Изд. 3-е, испр. и доп. - М.: Издательство Московского университета, 2005. - 448 с
11. Science Advances[Электронный ресурс] - Режим доступа: <https://advances.sciencemag.org/content/1/8/e1500363.full>
12. Unity User Manual [Электронный ресурс] - Режим доступа: <https://docs.unity3d.com/Manual/nav-InnerWorkings.html>

Приложение А. Техническое задание



МИНОБРНАУКИ РОССИИ
федеральное государственное бюджетное образовательное
учреждение высшего образования

«Национальный исследовательский университет «МЭИ»

Институт

ИВТ

Кафедра

ВМСС

ЗАДАНИЕ

НА ВЫПУСКНУЮ КВАЛИФИКАЦИОННУЮ РАБОТУ (бакалаврскую работу)

Направление

09.03.01 Информатика и вычислительная техника

(код и наименование)

Направленность (профиль)

Вычислительные машины, комплексы,

системы и сети

Форма обучения

очная

(очная/очно-заочная/заочная)

Тема: Разработка имитационной модели эволюции простейших организмов

Студент

А-07-16

Голикова А.В.

группа

подпись

фамилия и инициалы

Научный

к.т.н.

доцент

Раскатова М.В.

руководитель

уч. степень

должность

подпись

фамилия и инициалы

Зав. кафедрой

к.т.н.

доцент

Вишняков С.В.

уч. степень

звание

подпись

фамилия и инициалы

СОДЕРЖАНИЕ РАЗДЕЛОВ ЗАДАНИЯ И ИСХОДНЫЕ ДАННЫЕ

Необходимо разработать модель, позволяющую имитировать процесс взаимодействия клеток с окружающим их миром и между собой. Назначение приложения - наглядная демонстрация процесса эволюции простейших организмов для учащихся школ на уроках биологии.

Приложение позволяет пользователю наблюдать за передвижением и развитием клеток, отслеживать их текущее количество, изменение его со временем и типы организмов, присутствующих на поле. Также пользователь должен иметь возможность задавать количество еды, появляющейся на поле в определенный промежуток времени, начальное количество организмов и вероятность мутаций. Организмы могут получать энергию из пищи и тратить ее на передвижение или деление. В процессе деления могут происходить мутации с вероятностью, заданной пользователем. Как только у организма кончается энергия, он умирает.

Для создания приложения использовать среды разработки Visual Studio, Unity, язык программирования C#

Разработать удобный графический пользовательский интерфейс. Разработать программные модули управления взаимодействиями организмов между собой и с окружающей средой.

ПЕРЕЧЕНЬ ГРАФИЧЕСКОГО МАТЕРИАЛА

Количество листов	4
Количество слайдов в презентации	6
1. Схема алгоритма действий клетки	
2. Меню настроек симуляции	

3. Пользовательский интерфейс
4. График зависимости количества клеток разных типов от времени симуляции

РЕКОМЕНДУЕМАЯ ЛИТЕРАТУРА

1. Шилдт, Г. Полный справочник по C# : пер. с англ. / Г. Шилдт . – М. : Вильямс, 2004 . – 752 с.
2. Goldstone, W. Unity Game Development Essentials / Will Goldstone. – В. : Packt Publishing, 2009. – P. 316
3. Blackman, S. Beginning 3D Game Development with Unity/ Sue Blackman – В. : Apress, 2011. – P. 992

Примечания:

1. Задание брошюруется вместе с выпускной работой после титульного листа (страницы задания имеют номера 2, 3).
2. Отзыв руководителя, рецензия(и), отчет о проверке на объем заимствований и согласие студента на размещение работы в открытом доступе вкладываются в конверт (файловую папку) под обложкой работы.

Приложение Б. Листинг программы

```
using System.Collections;
using UnityEngine.UI;
using System.Collections.Generic;
using System;
using UnityEngine;
using UnityEngine.AI;

/*Скрипт, отвечающий за поведение сферы - передвижение, столкновение с другими игровыми
объектами, проверку энергии раз в 10 секунд, размножение или смерть.
Разработала: Голикова А.В. группа А-07-16
дата написания 01.05.2020
Версия 4
*/

public class sphere : MonoBehaviour {
    [SerializeField]
    public float speed=20;//скорость сферы
    public float size = 1;//размер сферы
    public float sense = 5;//размер сферы видимости
    public float energy = 20;//начальное количество энергии
    public int vegOrCar = 50;//начальный уровень хищничества
    private float basen = 159.0f;//базовая единица энергии, получаемой из пищи
    public GameObject meat;//физический объект "мясо", оставляемый сферой после смерти
    public GameObject eyes;//область видимости
    public Vector3 navigation;//вектор назначения
    public bool AchievedGoal=true;
    public vision goal;

    //процедура размножения сферы
    private void breed()//создание дочернего объекта
    {
        //создаем новую сферу и записываем ее компоненты в отдельную переменную для улучшения
        //производительности
        GameObject ChildBall = Instantiate (this.gameObject, new
        Vector3(this.gameObject.transform.position.x + 1, 1, this.gameObject.transform.position.z +
        1), Quaternion.identity);
        sphere childSphereComponents = ChildBall.GetComponent<sphere>();
        //энергия и начальная цель
        childSphereComponents.energy = 20;
        childSphereComponents.AchievedGoal = true;
        /*Мутации*/
        int NewMutation = UnityEngine.Random.Range(0, 9999);//от 0000 до 9999
        //скорость
        if (GameController.gc.mutationRate >= NewMutation/1000)//если первая цифра случайного
        числа в промежутке, нужном для мутации
        {
            childSphereComponents.speed += UnityEngine.Random.Range(-0.5f,0.5f);//изменить
            скорость
            if (childSphereComponents.speed < 0) childSphereComponents.speed = 0;//скорость не
            может быть меньше 0
        }
        //размер и масса
        if (GameController.gc.mutationRate >= NewMutation % 10)//если четвертая цифра
        случайного числа в промежутке, нужном для мутации
```



```

        {
            childSphereComponents.size += UnityEngine.Random.Range(-0.5f, 0.5f);
            if (childSphereComponents.size < 0.5) childSphereComponents.size = 0.5f;//>=0.5
            else if (childSphereComponents.size > 2) childSphereComponents.size = 2;//не
больше 2
        }
        //зрение
        if (GameController.gc.mutationRate >= NewMutation % 1000/100)//если вторая цифра
случайного числа в промежутке, нужном для мутации
        {
            childSphereComponents.sense += UnityEngine.Random.Range(-0.5f, 0.5f);//изменить
размер области видимости
            if (childSphereComponents.sense <= childSphereComponents.size)
childSphereComponents.sense = 0;//сфера может быть слепой, если радиус обзора меньше радиуса
сферы
        }
        //уровень хищничества
        if (GameController.gc.mutationRate >= NewMutation % 100 / 10)//если третья цифра
случайного числа в промежутке, нужном для мутации
        {
            childSphereComponents.vegOrCar += UnityEngine.Random.Range(-5, 5);
            if (childSphereComponents.vegOrCar > 100) childSphereComponents.vegOrCar =
100;//<=100
            else if (childSphereComponents.vegOrCar < 0) childSphereComponents.vegOrCar =
0;//>=0
        }
        //задаем внешний вид и другие параметры
        childSphereComponents.eyes.GetComponent<MeshRenderer>().transform.localScale= new
Vector3(childSphereComponents.sense, childSphereComponents.sense,
childSphereComponents.sense);
        ChildBall.GetComponent<MeshRenderer>().transform.localScale = new
Vector3(childSphereComponents.size, childSphereComponents.size, childSphereComponents.size);
        //объединяем сферы в одну папку BALLS для удобства

ChildBall.gameObject.transform.SetParent(GameObject.Find("BALLS").gameObject.transform);
    }

    //проверка энергии в конце дня
    private void check()
    {
        //вычисляем затраты энергии
        float Ediff = 0.9982f * 4 / 3 * 3.1415f * (float)Math.Pow(size, 3) * (1 +
(float)Math.Pow(speed, 2) / 2) + 0.73f * sense;
        energy -= Ediff;// уменьшаем энергию
        if (energy <= 0)//не хватило энергии
        {
            Destroy(this.gameObject);//уничтожить сферу
            GameObject NewFood=Instantiate(meat, new
Vector3((this.gameObject.transform.position.x + UnityEngine.Random.Range(-2.0f, 2.0f)), 0.5f,
(this.gameObject.transform.position.z + UnityEngine.Random.Range(-2.0f, 2.0f))),
Quaternion.identity);
            // объединяем в одну папку FOOD для удобства

NewFood.gameObject.transform.SetParent(GameObject.Find("FOOD").gameObject.transform.gameObject
.transform);

        }
        else if ((energy>=1.5f*Ediff) &&(GameController.gc.numOfBalls < 60))//может ли
размножиться, если сфер не слишком много
        {
            energy= energy- 1.5f * Ediff;
            breed();//процедура размножения
        }
    }

```

```

    }

    //столкновение с другим объектом
    private void OnTriggerEnter(Collider other)
    {
        switch (other.gameObject.tag)
        {
            case "Ball":
            {
                if ((this.size > 1.5 * other.GetComponent<sphere>().size) &&
                    (this.vegOrCar > 50))//поедание другой сферы
                {
                    Destroy(other.gameObject);
                    energy += other.GetComponent<sphere>().energy;
                    AchievedGoal = true;
                }
                break;
            }
            case "Vegetable"://поедание овоща
            {
                if (this.vegOrCar < 100)//если не абсолютный хищник
                {
                    Destroy(other.gameObject);//уничтожить овощ
                    energy += basen * (100.0f - vegOrCar) / 100.0f;//добавить энергию
                    AchievedGoal = true;
                }
                break;
            }
            case "Meat"://вижу мясо
            {
                if (this.vegOrCar > 0)//если не абсолютный вегетерианец
                {
                    Destroy(other.gameObject);//уничтожить мясо
                    energy += 6.0f * basen * vegOrCar / 100.0f;//добавить энергию
                    AchievedGoal = true;
                }
                break;
            }
            default:
                break;
        }
    }

    //инициализация сферы
    private void Start()
    {
        this.GetComponent<MeshRenderer>().materials[0].color = new Color32(Convert.ToByte(255
* this.vegOrCar / 100), 0, Convert.ToByte(255 - 255 * this.vegOrCar / 100), 255); //col;
        InvokeRepeating("check", 10, 10); //запуск проверки энергии каждые 10 секунд
        goal = eyes.GetComponent<vision>(); //запомнить компоненты зрения
        //задать скорость
        NavMeshAgent agent = GetComponent<NavMeshAgent>();
        agent.speed = speed;
    }

    //обновление симуляции
    void Update () {
        navigation = goal.navigation; //получить цель от области видимости
        //задать цель
        NavMeshAgent agent = GetComponent<NavMeshAgent>();
    }

```

```

        agent.destination = navigation;
        //если достиг цели, изменить ее на случайную
        if ((Math.Round(navigation.x) == Math.Round(this.gameObject.transform.position.x))&&
(Math.Round(navigation.z) == Math.Round(this.gameObject.transform.position.z)))
        {
            goal.navigation = new Vector3(UnityEngine.Random.Range(-40, 40), 0.5f,
UnityEngine.Random.Range(-20, 20)); //цель - случайная точка
        }
    }
}

```